

RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

Correctness-Aware Self-Healing for ML Forecasting Services on Kubernetes: Detecting and Recovering from Silent Failures

AIML-PROJECT REPORT

Submitted by

[STUDENT NAME 1] **[USN]**

[STUDENT NAME 2] **[USN]**

Under the guidance of

Dr. Merin Meleet
Associate Professor
Dept. of ISE
RV College of Engineering

In partial fulfillment for the award of degree of

Bachelor of Engineering

in

~~Information Science and Engineering~~

2025-2026

RV COLLEGE OF ENGINEERING®, BENGALURU - 560059

(Autonomous Institution Affiliatedto VTU, Belagavi) DEPARTMENT OF

INFORMATION SCIENCE AND ENGINEERING

CERTIFICATE

Certified that the project work titled "Correctness-Aware Self-Healing for ML Forecasting Services on Kubernetes: Detecting and Recovering from Silent Failures" is carried out by [STUDENT NAME 1] ([USN]) and [STUDENT NAME 2] ([USN]), who are bonafide student of R.V College of Engineering, Bangalore, in partial fulfillment for the award of degree of Bachelor of Engineering in Information Science and engineering of the Visvesvaraya Technological University, Belgaum during the year 2025-26. It is certified that all corrections/suggestions indicated for the internal Assessment have been incorporated in the report deposited in the departmental library. The project report has been approved as it satisfies the academic requirements in respect of project work as a part of the course "Artificial Intelligence and Machine Learning" prescribed by the institution for the said degree.

Faculty in Charge **Head of the Department**

External Evaluation

Name of Examiners **Signature with date 1**

2

RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

DECLARATION

We, [STUDENT NAME 1] and [STUDENT NAME 2], students of fifth semester B.E., Department of Information Science and Engineering, RV College of Engineering, Bengaluru, hereby declare that the project titled "Correctness-Aware Self-Healing for ML Forecasting Services on Kubernetes: Detecting and Recovering from Silent Failures" has been carried out by us and submitted in partial fulfilment for the award of degree of Bachelor of Engineering in Information Science and Engineering during the year 2025-2026.

Further we declare that the content of the dissertation has not been submitted previously by anybody for the award of any degree or diploma to any other university.

We also declare that any Intellectual property rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and we will be among the authors of the same.

_____

Place: Bengaluru

Date:

Signature

ACKNOWLEDGEMENTS

We express our sincere gratitude to our guide, Dr. Merin Meleet, Associate Professor, Department of Information Science and Engineering, RV College of Engineering, for her constant guidance, encouragement and valuable suggestions throughout this project. We thank the Head of the Department of Information Science and Engineering and the faculty of the department for providing the laboratory facilities and an environment that made this work possible. We are grateful to the Principal of RV College of Engineering for the academic support. We also thank the authors of the open-source tools on which this work stands - Docker, Kubernetes, Minikube, Prometheus, Grafana, OpenTelemetry, Jaeger, Argo Rollouts, Kyverno, cosign, KEDA, Chaos Mesh, Flask, TensorFlow, statsmodels, scikit-learn and kopf - and the authors of the research papers whose work we reviewed. Finally, we thank our families and friends for their patience and support during long experiment runs.

ABSTRACT

CONTENTS

Abstract	**i**
List of Figures	**v**
List of Tables	**vi**
List of Abbreviations	**vii**
Publication Details	**viii**
Chapter 1 Introduction	**1**
1.1 Terminology	**2**
1.2 Scope and Relevance	**3**
1.3 Motivation	**3**
1.4 Problem Statement	**4**
1.5 Objectives	**4**
1.6 Report Organisation	**5**
1.7 Summary	**5**
Chapter 2 Literature Survey	**6**
2.1 Literature Review	**6**
2.2 Summary of the Reviewed Papers	**8**
2.3 Research Gap and Positioning	**10**
2.4 Summary	**11**
Chapter 3 Software Requirements Specification	**12**
3.1 Functional Requirements	**12**
3.2 Non-Functional Requirements	**12**
3.3 Hardware Requirements	**13**
3.4 Software Requirements	**13**
3.5 Summary	**13**
Chapter 4 Design of the System	**14**
4.1 Theory and Concepts	**14**
4.2 Dataset Description	**16**
4.3 Design and Methodology	**18**
4.4 System Architecture	**20**
4.5 Tools and APIs	**22**
4.6 Summary	**22**
Chapter 5 Implementation and Testing	**24**
5.1 Implementation Requirements	**24**
5.2 Implementation Tool Features	**24**
5.3 Code Snippets with Explanation	**25**
5.4 Testing	**29**
5.5 Summary	**31**
Chapter 6 Results and Analysis	**32**
6.1 Results	**32**
6.2 Benchmarking and Analysis	**34**
6.3 Screenshots and Live Plots	**37**

6.4 Contributions (gap-based) 44
6.5 Deadline- and Load-Aware Degradation (Novelty 6) 45
6.6 Metamorphic Readiness Probe (experimental) 46
6.7 Baselines and Cost-Coverage of the Guard Layers 47
6.8 Healthy-but-Wrong Time and Mean Time to Correctness 48
6.9 Compound Faults 49
6.10 Overall Analysis 49
6.11 Summary 50
Chapter 7 Conclusion 51
7.1 Conclusion 51
7.2 Limitations 52
7.3 Future Scope 52
7.4 Summary 53
References (Ieee Format) 54
Appendix A Dataset Sample 57
Appendix B Research Paper (Ieee Format) 58

LIST OF FIGURES

Figure 4.1 Monthly rainfall, temperature and humidity (1951-2024) from data.csv 17
Figure 4.2 Monthly climatology used by the climatology tier 18
Figure 4.3 Methodology of the study 18
Figure 4.4 System architecture 21
Figure 4.5 Tiered degradation and guard checks 21
Figure 6.1 Wrong answers served per fault and configuration 33
Figure 6.2 Time to full quality under the operator 35
Figure 6.3 Home page of the Bangalore Climate Analytics System (browser, localhost:5000) 38
Figure 6.4 Climate dashboard: current values, 6-month forecast table and recent predictions 38
Figure 6.5 Dashboard forecast plots (rainfall and temperature) 39
Figure 6.6 Twelve-month SARIMA forecast plots served by the inference tier 39
Figure 6.7 Analysis gallery: distributions, anomaly z-scores, anomaly evolution and decade-wise anomalies 40
Figure 6.8 Live 12-step forecasts of SARIMA and LSTM returned by the deployed API 40
Figure 6.9 Live Grafana dashboard, last 30 minutes (refresh 5 s) 41
Figure 6.10 Grafana dashboard panels during the verification run 41
Figure 6.11 Golden-output integrity violations after an injected bit flip (Grafana panel) 42
Figure 6.12 Prometheus: golden_violation_total for the LSTM backend (1 h) 42
Figure 6.13 Prometheus: valid answers served per tier (1 h) 43
Figure 6.14 Jaeger trace of GET /dashboard with three inference calls 44
Figure 6.15 Jaeger trace search for the frontend service 44
Figure 6.16 Delay fault with and without deadline-aware degradation (10 trials each) 46
Figure 6.17 Detection of injected faults: baselines versus guard layers 48

LIST OF TABLES

Table 1.1 Terminology 3

Table 2.1 Reviewed papers (A1-A23): problem, method, result, limitation and relevance 9

Table 2.2 Gaps in the literature, what this project does, and the closest prior work 10

Table 3.1 Functional requirements 12

Table 3.2 Hardware requirements 13

Table 3.3 Software requirements 13

Table 4.1 Summary statistics of the daily data (26,835 rows) 17

Table 4.2 Configurations 19

Table 4.3 Injected faults (SilentBench and extensions) 19

Table 4.4 Inference API 22

Table 4.5 Configuration variables 22

Table 5.1 Implementation tools and the feature used 25

Table 5.2 Executed validation checks (19 of 19 passed) 30

Table 5.3 Defects found and fixed 31

Table 6.1 Baseline fault trials (mean over trials; recovery - means never recovered) 32

Table 6.2 Wrong answers served (percent of responses in the fault window), mean [95 % CI], n trials 33

Table 6.3 Which guard layer catches which fault (10 trials per cell) 34

Table 6.4 Operator policies (60 + 20 trials) 35

Table 6.5 Validation of tuned probe settings (hang fault) - a negative result for probe tuning 36

Table 6.6 Shadow gate decisions (selected candidates; 15 of 15 bad rejected, 6 of 6 good promoted overall) 37

Table 6.7 Additional components and their measured outcome 37

Table 6.8 Innovative components 45

Table 6.9 Metamorphic probe: detection per fault (no golden output, no stored reference) 47

Table 6.10 Detection of injected faults by simple baselines and by the layers of the guard (percent of windows) 47

Table 6.11 Compound faults: wrong answers and p95 latency per configuration 49
Table A.1 First 15 rows of data.csv 57
Table A.2 Ten rows from July 2018 57

LIST OF ABBREVIATIONS

AIML	-	Artificial Intelligence and Machine Learning
API	-	Application Programming Interface
ARIMA	-	Autoregressive Integrated Moving Average
CI/CD	-	Continuous Integration / Continuous Deployment
CPU	-	Central Processing Unit
CRD	-	Custom Resource Definition
DevOps	-	Development and Operations
DoE	-	Design of Experiments
HPA	-	Horizontal Pod Autoscaler
HTTP	-	Hypertext Transfer Protocol
IEEE	-	Institute of Electrical and Electronics Engineers
JSON	-	JavaScript Object Notation
K8s	-	Kubernetes
KEDA	-	Kubernetes Event-Driven Autoscaling
LFS	-	Large File Storage (Git)
LSTM	-	Long Short-Term Memory
ML	-	Machine Learning
MLOps	-	Machine-Learning Operations
MTTC	-	Mean Time To Correctness
MTTR	-	Mean Time To Recovery
PSI	-	Population Stability Index
RL	-	Reinforcement Learning
RSM	-	Response Surface Methodology
SARIMA	-	Seasonal Autoregressive Integrated Moving Average
SLI	-	Service-Level Indicator
SLO	-	Service-Level Objective
UCB	-	Upper Confidence Bound
YAML	-	YAML Ain't Markup Language

CHAPTER 1: INTRODUCTION

Machine-learning models are no longer research artefacts that are run once on a notebook. They are deployed as long-running network services that other programs, dashboards and people call thousands of times a day. Modern operations practice (DevOps) has produced a mature toolbox for keeping such services alive: containers package the code and its dependencies, Kubernetes schedules the containers and restarts them when they fail, and monitoring stacks such as Prometheus and Grafana make the behaviour of the running system visible. These tools were designed for ordinary web services, where a failure normally means that the service stops answering, answers with an error code, or answers too slowly.

A machine-learning service can fail in a way that none of these tools notices. If the weights of a neural network are damaged, if an input scaler is replaced by one trained on different units, if a cache returns a stale forecast, or if a data pipeline silently changes the unit of a feature, the service continues to answer every request promptly with HTTP status 200 and a perfectly well-formed JSON document. The numbers inside the document, however, are wrong. We call this class of problem a silent failure. Because the process is running, the port is open and the response code is 200, the liveness and readiness probes that Kubernetes and Docker Compose use report the service as healthy, and the platform never intervenes. The users, or a downstream system that trusts the forecast, simply receive wrong information for as long as the fault lasts.

This project studies silent failures in a concrete and realistic setting: the Bangalore Climate Analytics System, a Flask web application that forecasts monthly rainfall, mean temperature and humidity for the city of Bengaluru from 74 years of daily weather records (1951-2024) using two forecasting models, a seasonal ARIMA (SARIMA) model and a long short-term memory (LSTM) neural network. The original application is a single-process monolith. We re-engineer it as a containerised, Kubernetes-hosted system with a separate frontend, an inference service and a PostgreSQL database, and then use it as a test bed to answer a research question that is not answered by the standard DevOps toolbox: how can an orchestrated machine-learning service detect that its own answers are wrong, keep serving safe answers while the fault lasts, restore full quality automatically, and even predict some failures before they happen?

The contribution of the project is therefore twofold. First, it provides a measured characterisation of the silent-failure blind spot in default monolith, Docker Compose and Kubernetes deployments, using a purpose-built silent-fault benchmark (SilentBench) with eight injected fault types. Second, it designs, implements and evaluates a layered set of mechanisms that close the blind spot: a conformal-prediction based correctness guard, a golden-output integrity check, a cross-model canary vote, tiered graceful degradation, deadline- and load-aware degradation, a correctness service-level objective (SLO), a custom Kubernetes operator that restores full quality, a failure-precursor predictor, and a shadow-traffic gate for model delivery. Every claim made in this report is backed by an executed experiment whose raw data are stored with the project, and limitations are stated explicitly.

1.1 Terminology

The following terms are used throughout the report. They are defined once here so that the later chapters can use them without repeated explanation.

Term	Meaning in this project
Silent failure	A fault in which the service keeps answering with HTTP 200 and well-formed output, but the numerical content is wrong (for example corrupted weights, wrong scaler, stale data).
Liveness probe	A Kubernetes check that decides whether a container must be restarted. By default it tests only that the process answers a trivial request.
Readiness probe	A Kubernetes check that decides whether a pod receives traffic. By default it tests only that the application has started.
Semantic health	Health defined by the correctness of the model output, not by process liveness. Exposed by the service on /health/semantic.
Conformal envelope	A distribution-free acceptance region for forecasts, calibrated on historical windows, that bounds the false-alarm probability by the chosen level alpha.
Window-level calibration	Calibrating the envelope on whole forecast windows (family-wise) instead of single values, so that the false-alarm rate refers to a complete forecast.
Invariant	A property that every correct forecast must satisfy, for example that mean temperature is not constant (flat-line) across the forecast horizon.
Cross-model canary	A background task that runs all models (SARIMA, LSTM, climatology) on the same horizon, aligns them by calendar month and marks a model suspect when it disagrees with the others.
Golden output	The forecast produced by each model at start-up while it is known to be healthy; later outputs must reproduce it (models are deterministic).
Tier	One level of the degradation chain: LSTM, SARIMA, climatology (monthly mean), cache (last valid forecast).
Graceful degradation	Serving a lower-quality but still valid answer from a lower tier instead of failing or serving a wrong answer.
Correctness SLO	A service-level objective stated on the fraction of served answers that are valid, estimated with a Wilson lower confidence bound and monitored with burn rates.
MTTR / restoration time	Time from fault injection until full-quality answers are served again.
Operator / CRD	A Kubernetes controller (built with kopf) that watches a custom resource, ForecastService, and takes recovery actions.
Digital twin / bandit	A table of expected recovery cost per fault signature and action, learned from experiments, and an upper-confidence-bound bandit that chooses actions from it.
Shadow gate	Evaluation of a candidate model on live traffic without serving its answers; the candidate is promoted only if its reject ratio is small.

Term	Meaning in this project
Deadline-aware degradation	A per-request time budget and per-tier concurrency caps; a tier that cannot answer in time is skipped so that a cheaper tier answers.
SilentBench	The silent-fault benchmark designed in this project: eight fault types injected into four configurations, 5-10 trials per cell.
Healthy-but-wrong	The state in which the platform reports the service healthy while it serves incorrect answers.

Table 1.1 Terminology

1.2 Scope and Relevance

The scope of the project covers the complete life cycle of a machine-learning inference service on a container platform: packaging (Docker), orchestration (Kubernetes on Minikube), observability (Prometheus, Grafana, OpenTelemetry with Jaeger), fault injection (custom chaos drivers, Chaos Mesh), automatic recovery (probes, a custom operator), and delivery (shadow-traffic gate, Argo Rollouts, signed images verified by an admission controller). The workload is a time-series forecasting service, but the mechanisms are described in terms of generic model outputs and can be applied to other regression-type services.

The relevance is practical and academic. Practically, organisations increasingly run models behind Kubernetes services, and a wrong forecast that is served with a healthy status can cause real harm: a wrong rainfall forecast can mislead a flood-preparedness dashboard, and a wrong demand forecast can mislead an autoscaler that then provisions the wrong capacity. Academically, the literature on Kubernetes resilience concentrates on infrastructure faults (pod kills, node failures, latency) and on anomaly detection in infrastructure metrics, logs and traces, while the correctness of the model output is treated as outside the scope of the platform. Our review of twenty-three recent IEEE papers (Chapter 2) found that none of them evaluates correctness-aware health checking for ML outputs, which is the gap this work addresses.

The study is deliberately bounded. It runs on a single Windows laptop with a one-node Minikube cluster, uses 5-10 trials per experimental cell, and studies one model family on one data set. It does not claim production-grade security (Appendix findings in the security review) and does not attempt multi-node scaling conclusions. These bounds are repeated in Chapter 7.

1.3 Motivation

The motivation came from a simple observation made while containerising the original application. After the monolith was split into a frontend and an inference service and

deployed on Kubernetes, a fault-injection endpoint was used to corrupt the LSTM weights. The pod stayed Running, the readiness probe stayed green, Grafana showed healthy request rates and latency, and every response was HTTP 200 - yet the forecasts were nonsense. The platform that was supposed to provide self-healing had no way of knowing that anything was wrong.

Several further observations strengthened the motivation. Reliability engineering practice defines service-level objectives on availability and latency, not on correctness, so error budgets are not consumed by wrong-but-fast answers. Existing anomaly detectors watch infrastructure signals, which are normal during a silent failure. Model-delivery practice evaluates candidate models offline, not as part of the deployment pipeline. And the few papers that combine chaos engineering with learning (for example reinforcement-learning-guided fault orchestration) optimise infrastructure recovery rather than output correctness. A systematic, measured study of silent failures in orchestrated ML services therefore seemed both necessary and feasible on modest hardware.

1.4 Problem Statement

Given a machine-learning forecasting service deployed on Kubernetes, design and evaluate a set of mechanisms that (a) detect silent failures, in which the service returns well-formed but incorrect forecasts, without relying on the model's own confidence; (b) keep the service answering with valid, clearly labelled answers while the fault persists; (c) restore full-quality answers automatically and quickly; (d) anticipate selected failures from their precursors; and (e) protect the delivery path so that a degraded model version is not promoted - while generating no false alarms on healthy operation and keeping the added latency and complexity acceptable.

1.5 Objectives

- O1. Re-engineer the monolithic Bangalore Climate Analytics application (SARIMA + real trained LSTM) into containerised services without changing the meaning or output of the original models.

- O2. Build a silent-fault benchmark (SilentBench) with eight injected faults and measure how monolith, Docker Compose and default Kubernetes deployments behave under it.

- O3. Design a correctness guard that detects silent failures with a statistically controlled

false-alarm rate (conformal envelope, invariants, cross-model canary, golden-output check).

- O4. Implement tiered graceful degradation and a deadline- and load-aware variant so that the service never serves a wrong answer and rarely fails a request.
- O5. Define and monitor a correctness SLO and expose live metrics, dashboards and traces (Prometheus, Grafana, OpenTelemetry/Jaeger).
- O6. Implement a Kubernetes operator (CRD ForecastService) that restores full quality, and compare rule-based and learned (digital-twin + bandit) recovery policies.
- O7. Evaluate probe tuning (design of experiments and Bayesian optimisation), a failure-precursor predictor and a shadow-traffic delivery gate.
- O8. Report all results with confidence intervals and exact tests, state every limitation honestly, and package data, code and documents so that the work can be reproduced and written up as a research paper.

1.6 Report Organisation

Chapter 2 surveys the literature and positions the work. Chapter 3 lists the functional, hardware and software requirements. Chapter 4 explains the underlying theory, the data set, the design methodology and the system architecture. Chapter 5 describes the implementation, key code and the testing strategy. Chapter 6 presents the results, benchmarking, screenshots and the innovative components. Chapter 7 concludes with limitations and future scope. The appendices contain a data sample and a condensed research paper in IEEE style.

1.7 Summary

This chapter introduced the problem of silent failures in orchestrated machine-learning services, defined the terminology, delimited the scope, explained the motivation and stated the problem and the objectives. The remainder of the report documents how the problem was characterised, how it was solved with a layered set of mechanisms, and how well the solution works.

CHAPTER 2: LITERATURE SURVEY

This chapter reviews the literature that surrounds the project. Twenty-three recent papers, supplied as a reading list from IEEE sources, were read in full-text form and organised by theme: chaos engineering and resilience on Kubernetes, anomaly detection for microservices, failure prediction, predictive autoscaling, monolith-to-microservice migration, SLO-based reliability practice, time-series forecasting and conformal prediction. For each paper we record the problem, the method, the key result, the limitation and the relevance to this project. A keyword scan of the full text of every paper was used to check, rather than assume, which concepts they do and do not contain.

2.1 Literature Review

2.1.1 Chaos engineering and resilience on Kubernetes

Siwach and Haridas evaluate the operational readiness of a Big-Data Kubernetes deployment by killing pods at random and observing recovery time. The study establishes the basic experimental pattern that this project also uses (inject, observe, measure recovery), but its faults are infrastructure faults and the workload has no model output whose correctness could be examined. Nigam and Patel propose AI-augmented chaos testing, combining a graph neural network that scores component criticality, a transformer that forecasts anomalies, reinforcement learning that orchestrates experiments and a digital twin (the Resilience Mirror); they report up to 46 percent fewer SLO violations and 38 percent lower MTTR. Zaccarini et al. (TELKA) use chaos engineering, reinforcement learning and a digital twin to reschedule pods. Both papers show that a learned policy plus a digital twin is an accepted idea, which is why this project treats its digital-twin and bandit policy as a supporting component rather than as a claimed novelty. Neither paper examines whether the output of an ML workload is correct.

2.1.2 Anomaly detection for microservices

A large group of papers detect anomalies in infrastructure signals: AnoTraceAE uses a convolutional autoencoder on distributed traces; TraceDAE uses a dual autoencoder over a spatio-temporal graph of traces; VTMAD combines a variational graph autoencoder with a transformer over multi-modal data; MicroMILTS locates faulty services with mutual information and an LSTM autoencoder; a multi-level ensemble detector combines statistics, Isolation Forest and adaptive thresholds on Prometheus

metrics; LogBERT-Lite applies a transformer autoencoder to logs; SGCAD uses structure-aware neural Granger causality; and a hybrid autoencoder-LSTM framework scores multi-cloud traffic. These methods detect that a service or a call path behaves abnormally. During a silent failure, however, request rates, latencies, traces and logs are all normal, so detectors of this kind have nothing to detect. This project verifies the point experimentally: a log-based Isolation-Forest detector found no log templates and detected none of six silent faults (Section 6.2.7).

2.1.3 Failure prediction

Wu et al. predict service reliability with an LSTM and a Bayesian network; Sun et al. use a stacking ensemble for failure prediction in optical networks with imbalanced data; Tufeanu et al. predict time-to-collapse with machine learning in a 5G/6G Kubernetes service-based architecture and drive garbage-collection agents. These works justify using a learned early-warning model; this project applies the idea to precursors specific to model-serving processes (memory growth and distribution drift) and uses an Isolation Forest plus a Mahalanobis distance trained on healthy telemetry.

2.1.4 Predictive autoscaling

Li proposes a PathFormer-based CPU forecast that drives the number of replicas through Prometheus and the Kubernetes API and reports better behaviour than the Horizontal Pod Autoscaler on public traces. The method ignores model warm-up time, which for a TensorFlow inference pod is about 4.6 seconds of process start-up plus image and model loading. This observation motivated the warm-up-aware scaling experiments (KEDA, a reinforcement-learning simulation) in the extras of this project.

2.1.5 Monolith versus microservices

Berry et al. report an experience of migrating a monolith to microservices and compare performance, availability and energy; they find that microservices are preferable when nothing is replicated and the monolith is preferable when the whole application is replicated. Zairi and Kenzi review twenty decomposition studies, and Xue et al. describe a Kubernetes teaching platform. These papers justify the baseline configurations A (monolith), B (Docker Compose) and C (Kubernetes) used here and the energy-aware tiering extra.

2.1.6 Service-level objectives

Gummadi and Chilamkurti describe SLO, SLI and error-budget practice with Splunk

and Dynatrace for microservices: the objectives are about latency, availability and checkout-flow success, and the paper reports fewer alert storms. Correctness of a model output is not an objective in the paper. We define a correctness SLO whose SLI is the fraction of served answers that pass an independent validity check, estimate it with a Wilson lower bound and alert on multi-window burn rates.

2.1.7 Forecasting models and conformal prediction

Singh describes the deployment of SARIMA models for network reliability prediction, which confirms SARIMA as a suitable and interpretable baseline forecaster for seasonal series. Hajihashemi and Shen present enhanced multi-model online conformal prediction with model-subset selection; it is the only paper in the reading list that contains conformal prediction, and it is a theoretical contribution without a systems or health-checking application. It supports the technique used in this project but not its application to Kubernetes health checks.

2.2 Summary of the Reviewed Papers

#	Paper (authors, year)	Problem / method	Key result	Limitation / gap	Relevance to this project
A1	AI-Augmented Chaos Testing: Toward Intelligent Resilience Engineering in Cloud-Native Systems (S. Nigam, S. Patel, 2025 (title))	GNN criticality + transformer anomaly forecast + RL experiment orchestration + "Resilience Mirror" digital twin	up to 46 % fewer SLO violations, 38 % lower MTTR (Kubernetes, edge, GPU)	generic workloads; no ML-output correctness	closest chaos/RL work; cite for chaos + digital twin
A2	A Comparative Study of Automatic Migration from Monolith App to Microservices (J. Zairi, A. Kenzi, ~2025)	review of 20 decomposition studies	framework + criteria	no runtime data	migration background
A3	A Predictive Elastic Scaling Method for Kubernetes Based on Load Forecasting (L. Li, ~2025)	PathFormer CPU forecast → replicas via Prometheus/K8s API	beats HPA (ETTm2, Alibaba trace, cluster)	ignores model warm-up	autoscaling baseline
A4	A Stacking Ensemble ML-Based Failure Prediction Model for Optical Networks with Imbalanced Data (Z. Sun et al., ~2024)	stacking ensemble, 5.3 % failure ratio	F1 98.97 %	optical networks	imbalance handling idea
A5	AdaptiPID: Model-Aware Adaptive Gain Tuning … Robotic Manipulator Control (Y. Zhou, ~2026)	adaptive PID	–	unrelated	excluded
A6	Architecture-based Failure Prediction via LSTM and Bayesian Network in Service-oriented Systems (X. Wu et al., ~2022)	LSTM reliability series + BN	+14 % accuracy	infra series only	failure-prediction related work
A7	Autoencoder-based Anomaly Detection in Microservices using Distributed Tracing (AnoTraceAE) (not read from page 1, 2024 (2024 AISP header))	conv. autoencoder on traces, TrainTicket	superior to baselines	detection only	detector related work
A8	Chaos Engineering based Kubernetes Pod Rescheduling through Deep Sets and RL (M. Zaccarini, F. Poltronieri, D. Borsatti, W. Cerroni, L. Foschini, G. Grabarnik et al., ~2025)	TELKA: chaos + RL + digital twin for pod rescheduling	scalable to varying node counts	scheduling only	related to N3
A9	Deployment of SARIMA Models for Network	SARIMA deployment steps	short-term reliability	no ops metrics	SARIMA context

#	Paper (authors, year)	Problem / method	Key result	Limitation / gap	Relevance to this project
	Reliability Prediction (D. Singh, 2024)		prediction		
A10	Evaluating operational readiness using chaos engineering simulations on Kubernetes in Big Data (G. Siwach, A. Haridas, ~2022)	random pod kills, recovery time	recovery time inferred	no ML/silent faults	baseline chaos study
A11	Is it Worth Migrating a Monolith to Microservices? Performance, Availability and Energy (V. Berry et al., ~2024)	migration experience report	MSA better when nothing replicated; monolith better when entire app replicated	generic web app	closest migration study
A12	Kube-CC: Experiment Education Platform for Big Data Courses on Kubernetes (F. Xue et al., ~2023)	K8s teaching platform	–	no resilience data	background only
A13	MicroMILTS: Fault Location for Microservices (Mutual Information + LSTM-AE) (L. Yang et al., ~2022)	BIRCH + LSTM-AE + MI	locates faulty service	infra metrics	detector related work
A14	Multi-Level Ensemble Anomaly Detection for Microservices (Statistics, Isolation Forest, Adaptive Thresholding) (not read, ~2025)	Prometheus metrics, layered detector	stable detection on ERP-like testbed	no recovery	Isolation-Forest-on-metrics precedent
A15	Recall-Prioritized Anomaly Detection in Cloud Microservices using Unsupervised Transformer Autoencoders (A. C. et al., ~2025)	LogBERT-Lite on OpenStack logs	8/11 faults, precision 4.37 %	low precision	related work
A16	Research on Constraint-Aware Fault Recovery Diagnosis Decision Method for Offline Edge Kubernetes (Z. Wei, K. Bai, ~2025)	Repair/Degrade/Rebuild decision tree	MTTR 127 → 34 min (case)	manual-style, edge	"Degrade" strategy precedent for tiers
A17	Residual Artifact Governance With ML-Enabled GC Prediction in K8s 5G/6G SBA (L.-M. Tufeanu et al., 2026 (IEEE TNSM vol. 23))	ML time-to-collapse + GC agents	bounded residual peaks	telecom	ML early-warning analogy
A18	Robust Hybrid Autoencoder-LSTM Framework for Real-Time Anomaly Detection in Multi-Cloud (S. Oviya et al., ~2025)	AE + LSTM score	94.2 % acc, 5.6 % FP (CICIDS2017)	intrusion data	comparison detector
A19	SLO Observability with Splunk and Dynatrace in Microservices (V. P. K. Gummadi, L. S. Chilamkurti, ~2025)	SLO/SLI/error budgets practice	alert-fatigue reduction	no experiments	SLO framing for C1
A20	Multi-Modal Anomaly Detection for Microservices Based on VGAE-Transformer (VTMAD) (C. Shao, X. Zhou et al., ~2025)	VGAE + Transformer	outperforms baselines (2 datasets)	heavy, no healing	detector related work
A21	TraceDAE: Trace-Based Anomaly Detection in Microservice Systems via Dual Autoencoder (J. Li, S. Ying, T. Li et al., 2025 (IEEE TNSM vol. 22 no. 5))	STG + GAT/LSTM dual AE	efficient detection	trace anomalies only	detector related work
A22	Enhanced Multi-model Online Conformal Prediction (E. Hajihashemi, Y. Shen, not read)	online conformal, bipartite-graph model subset selection	smaller sets, lower cost	theory; no systems use	only paper with conformal in the zips; supports N1 technique, not novelty
A23	SGCAD: Structure-aware Granger Causality-based Anomaly Detection for Microservice Systems (Z. Zhang, C. Wan et al., not read)	neural Granger causality + root-cause ranking	high recall, low FP (RS-Anomic)	metrics only	detector related work

Table 2.1 Reviewed papers (A1-A23): problem, method, result, limitation and relevance

2.3 Research Gap and Positioning

The positioning of this work is gap-based: for each point we state what the surveyed papers do not do and what this project did. A full-text keyword scan of the twenty-three papers, extended by web searches for recent preprints, gave the following picture. Resilience and chaos-engineering papers inject infrastructure faults only; anomaly detectors operate on metrics, logs and traces, which are normal during a silent failure; SLO practice concerns latency and availability; readiness-probe guides for model servers test whether the server can answer, not whether the answer is correct; recovery work optimises infrastructure recovery; and delivery pipelines evaluate models offline.

Gap in the existing work	What this project does	Evidence	Closest prior work
Chaos studies inject infrastructure faults only	SilentBench: eight faults that keep the service healthy but wrong, on monolith, Compose and Kubernetes	Default probes: 100 % wrong answers, 0 detections (10 trials per cell)	Chaos papers (Siwach and Haridas; Nigam and Patel; Zaccarini et al.)
Detectors watch metrics, logs and traces, which are normal in a silent failure	Health defined on the model output; a log-based detector measured as blind	0 log templates, 0 of 6 faults detected from logs	Trace/log/metric anomaly detectors (A7, A13-A15, A18, A20, A21, A23)
SLOs cover latency and availability, not answer correctness	Correctness SLO with Wilson lower bound and burn rates	Alert 9-15 s after value corruption	Gummadi and Chilamkurti; a runtime SLO guard for ML serving (arXiv 2607.09992) is close prior work
Probe guides test "can serve", not "serves correct answers"; the recovery trade-off is unmeasured	Semantic readiness, semantic liveness and partial readiness measured under silent faults	Readiness alone: 92 % failed, never restored; readiness + liveness: 36 s; partial readiness 0 % vs strict 81 % failed	llm-d readiness guide; vLLM health-check issue (qualitative only)
Recovery work optimises infrastructure, not answer quality	Operator that restores full quality; cost model that charges time below full quality	20 of 20 restored in about 22-24 s vs never; cost-blind bandit 5 of 20	Chaos + RL + digital-twin papers (Nigam and Patel; Zaccarini et al.)
Delivery pipelines evaluate models offline	Shadow-traffic gate on output validity and an Argo canary driven by it	15 of 15 bad candidates rejected, 6 of 6 good promoted	Standard canary practice
Degradation and deadlines are described in practitioner write-ups, not measured on real faults	Deadline- and load-aware tier fall-through measured under a delay fault	Failed requests 96.8 % to 0 %	Edge failover study (2023) and fallback-cascade write-ups

Table 2.2 Gaps in the literature, what this project does, and the closest prior work

Several ingredients are individually known and are cited as such: golden-output and bit-flip detection (LM-Fix, RADAR, TrustSeed), conformal runtime monitoring of AI services (WATCH; evaluation-as-a-service with conformal guarantees; online conformal prediction by Hajihashemi and Shen), self-healing machine-learning frameworks, and tiered fallback. The contribution claimed here is therefore empirical

and systems-level: a measured study of silent ML failures across deployment styles and an integrated, Kubernetes-native defence with quantified recovery, not a new algorithm. Only the twenty-three papers and a limited web search were checked; a structured IEEE Xplore and Google Scholar search is required before the word first is used in a paper.

2.4 Summary

The reviewed literature offers strong tools for finding infrastructure anomalies, predicting infrastructure failures, scheduling pods and scaling replicas, but it treats the correctness of model outputs as out of scope. The gap is therefore not a missing detector for yet another infrastructure signal but a missing notion of health that is defined on the answer itself. Chapter 3 translates this positioning into requirements.

CHAPTER 3: SOFTWARE REQUIREMENTS SPECIFICATION

This chapter states what the system must do (functional requirements), what it needs to run (hardware and software requirements) and which quality attributes it must satisfy (non-functional requirements).

3.1 Functional Requirements

ID	Requirement	Verified by
FR1	The application shall let a user register, log in and request rainfall, temperature and humidity forecasts with SARIMA and LSTM for a chosen horizon.	V12, browser walk-through
FR2	The inference service shall expose /predict, /health/live, /health/ready, /health/semantic, /metrics, /slo, /aging, /drift, /energy and shadow endpoints.	V07, V10
FR3	The service shall never return a forecast that fails the correctness checks; it shall fall through the tier chain LSTM, SARIMA, climatology, cache.	V08, V11, SilentBench
FR4	Degraded answers shall be labelled (degraded flag, tier name) and shown to the user as a badge.	UI check e8
FR5	The system shall provide fault-injection endpoints for crash, hang, out-of-memory, corrupt, bit-flip, scaler, stale, unit-change, flat-line, delay, memory-leak, drift and missing-variable faults and a heal endpoint.	chaos drivers
FR6	The operator shall watch a ForecastService custom resource, evaluate /health/semantic and restart the deployment after a configurable number of consecutive invalid checks.	Phase 8, 60 trials
FR7	The system shall publish a correctness SLO (served valid / invalid, Wilson lower bound, burn rates) and Prometheus alert rules.	V13, Phase 7
FR8	The system shall predict memory-leak and drift failures from telemetry and report lead time.	Phase 10
FR9	A candidate model shall be evaluated on live traffic in shadow mode and promoted only if the reject ratio is at most 5 percent over at least 20 samples.	Phase 11
FR10	A per-request time budget and per-tier concurrency caps shall allow a slow tier to be skipped.	V19
FR11	Metrics, dashboards and traces shall be available in Prometheus, Grafana (15 panels) and Jaeger.	V13, browser

Table 3.1 Functional requirements

3.2 Non-Functional Requirements

- Correctness: zero wrong answers served under every injected silent fault when the guard is on.
- False alarms: no rejection of healthy forecasts in a ten-minute soak and on hold-out windows (design level alpha = 0.01).
- Recovery: full quality restored automatically within about 30 seconds of a silent fault under the operator.
- Performance: healthy median latency of a few tens of milliseconds for SARIMA and about 50-60 ms for LSTM; the guard must not add noticeable latency.

- Reproducibility: every experiment is a script, every result file is retained, including superseded and invalid runs.

- Portability: the system runs on a single laptop (Docker Desktop + Minikube) and can be restored from the repository (RESTORE.md).

- Honesty: negative and inconclusive results are recorded as such and not used as evidence.

3.3 Hardware Requirements

Item	Specification used
Processor	Multi-core x86-64 laptop CPU (Minikube node limited by Docker Desktop; 4 or more vCPUs recommended)
Memory	16 GB RAM recommended; Minikube/Docker VM given about 8 GB. The LSTM/TensorFlow pod requests 1 GiB and is limited to 2.5 GiB.
Disk	20 GB free (Docker images about 2.5 GB each for TensorFlow, SARIMA pickles of 118-124 MB per variable, Git LFS objects)
Network	Internet access for image pulls and Helm charts; none needed at run time
Power	Mains power and sleep disabled during long experiments (Docker Desktop stops when the laptop sleeps)

Table 3.2 Hardware requirements

3.4 Software Requirements

Component	Version / role
Operating system	Windows 11 with WSL2 backend for Docker Desktop
Docker Desktop	29.8.2 - container build and run, Docker Compose profiles
Minikube + kubectl	kubectl client 1.36 - single-node Kubernetes cluster, images loaded with minikube image load
Helm	Prometheus, Grafana, Kyverno, KEDA, Chaos Mesh charts
Python	3.10 - Flask, gunicorn (gthread), TensorFlow/Keras (LSTM), statsmodels (SARIMA), scikit-learn, scipy, pandas, numpy, matplotlib, kopf, requests
PostgreSQL	15-alpine - user accounts and forecast history (SQLite for the monolith)
Monitoring	Prometheus (scrape and rule evaluation 5 s), Grafana (dashboard climate-live), kube-state-metrics, metrics-server
Tracing	OpenTelemetry SDK 1.27 with OTLP/HTTP exporter, Jaeger all-in-one 1.57
Delivery and security extras	Argo Rollouts 1.7.2, Kyverno, cosign 2.4.1, local registry
Version control	Git with Git LFS (models), GitHub repository DEVOPS

Table 3.3 Software requirements

3.5 Summary

The functional requirements describe a forecasting application with a correctness-aware inference service, an operator, an SLO and delivery safeguards; the non-functional requirements fix the quality targets that Chapter 6 verifies; and the hardware and software lists show that the whole study can be reproduced on a single mid-range Windows laptop.

CHAPTER 4: DESIGN OF THE SYSTEM

This chapter explains the theory and technology on which the system is built, describes the data set, presents the design and the experimental methodology, and shows the system architecture together with the tools and application programming interfaces (APIs) that were used.

4.1 Theory and Concepts

4.1.1 Containers, Docker and Docker Compose

A container packages an application together with its libraries and configuration and runs it as an isolated process that shares the kernel of the host. Docker builds container images from a Dockerfile and runs them; Docker Compose describes a multi-container application (frontend, inference service, database) in one YAML file so that it can be started with a single command. In this project Compose is configuration B: it restarts a crashed container (restart policy) and can use a health check, but it has no notion of readiness gating or of correctness.

4.1.2 Kubernetes, probes and self-healing

Kubernetes (K8s) is a container orchestrator. A Deployment keeps a desired number of identical pods running; a Service gives them a stable address; the kubelet runs the probes. A liveness probe that fails repeatedly causes the container to be restarted; a readiness probe that fails removes the pod from the Service endpoints; a startup probe protects slow-starting containers. The default probes used with most images test only that an HTTP endpoint answers. This is exactly what a silent failure does not break, which is why the default self-healing of Kubernetes (configuration C) never reacts to it. Kubernetes can be extended with custom resources (CRDs) and controllers, called operators, that encode domain-specific recovery logic.

4.1.3 Observability: Prometheus, Grafana and OpenTelemetry

Prometheus periodically scrapes text-format metrics from the pods (counters, gauges, histograms) and evaluates alert and recording rules; Grafana draws live dashboards from Prometheus queries; OpenTelemetry instruments the code to emit distributed traces, which Jaeger stores and displays. In this project the inference service exports request counters, latency histograms, semantic scores, tier usage, correctness SLI counters, aging and drift gauges, energy estimates, golden-check violations and shed-

request counters, and the frontend exports hop-timing histograms.

4.1.4 Time-series forecasting: SARIMA and LSTM

A seasonal ARIMA model, written SARIMA(p,d,q)(P,D,Q)s, represents a series as an autoregressive and moving-average process after differencing, with an additional seasonal component of period s; for monthly climate data s equals 12. It is statistically interpretable and cheap to evaluate once fitted. An LSTM is a recurrent neural network with gated memory cells that can learn non-linear temporal dependencies; here a separate LSTM per variable forecasts the monthly series after min-max scaling. The two models have different failure modes (a damaged SARIMA pickle versus damaged LSTM weights or scaler), which is the basis for a cross-model vote.

4.1.5 Conformal prediction and the window-level envelope

Split conformal prediction turns any point predictor into a procedure with a distribution-free coverage guarantee: residuals on a calibration set define a quantile q, and under exchangeability a new residual exceeds q with probability at most alpha. The guard in this project uses the same idea as an acceptance test rather than a prediction interval. Historical forecast windows from years up to 2018 are used for calibration, and a scalar nonconformity score is computed for the entire window (family-wise), so that the probability that a healthy forecast is rejected is controlled at the window level. The score divided by the quantile gives a value that is at most 1 for a valid forecast, which is why the code compares the score with 1.0. The hold-out windows after 2018 are used to confirm the false-alarm rate.

4.1.6 Invariants, cross-model voting and golden outputs

Invariants are cheap necessary conditions: for example the mean-temperature forecast must not be constant (flat-line check) and must follow a plausible seasonal pattern. Cross-model voting runs all models on the same horizon, aligns them to the same calendar months and flags the model that disagrees with the others, in the spirit of N-version programming but with heterogeneous forecasters (SARIMA, LSTM, climatology). A golden-output check exploits determinism: the forecast of each model at start-up is stored while the model is known to be healthy, and any later forecast that deviates by more than 0.002 history-standard-deviations marks the model suspect. It detects subtle corruptions, such as a few flipped exponent bits, that keep the output inside the statistical envelope.

4.1.7 Service-level objectives and burn rates

An SLO states a target for a service-level indicator (SLI) over a window; the error budget is one minus the target; the burn rate is the speed at which the budget is consumed. Here the SLI is the fraction of answers that pass an independent validity check after serving. The lower end of the Wilson score interval is used so that a small number of requests does not give a falsely optimistic estimate, and alert rules compare fast and slow burn rates.

4.1.8 Recovery policies: rules, digital twin and bandit

A rules policy restarts the deployment after N consecutive invalid semantic checks. A digital twin in this project is a table, learned from experiments, of the expected cost of each action for each fault signature (unreachable, timeout, invalid models); cost combines recovery time, failed requests, degraded answers and, in the corrected version, the time spent below full quality. An upper-confidence-bound (UCB) bandit chooses the action with the lowest optimistic cost estimate and keeps exploring actions it has seen rarely.

4.1.9 Failure prediction and software aging

Long-running processes can age: memory grows, distributions drift. The predictor trains an Isolation Forest and a Mahalanobis distance on healthy telemetry windows (memory, CPU, latency, scores) and raises an alarm when a window is atypical. Aging is additionally summarised with a Weibull fit and drift with the population stability index (PSI) and the Page-Hinkley test.

4.1.10 Deadline-aware degradation

A request is given a time budget (800 ms). Each model tier is executed in a worker pool and protected by a bounded semaphore (two concurrent LSTM calls, three SARIMA calls). If the semaphore cannot be acquired or the result does not arrive before the deadline, the tier is skipped (counted as a shed request with reason load, deadline or timeout) and the next, cheaper tier answers. This is a form of load shedding that converts slowness into a labelled, lower-quality answer instead of a timeout.

4.2 Dataset Description

The data are the daily weather records of Bengaluru from 1 January 1951 to 20 June 2024, stored in data.csv with the columns Date, Rain (mm), Temp Max and Temp Min (degrees Celsius) and Humidity (percent). There are 26,835 daily rows. The training

pipeline aggregates them to monthly series (rainfall total, mean temperature as the average of daily maximum and minimum, mean humidity). The same data set (identical content) is used by the monolith, by the frontend (for its history and analysis pages and for the input guard) and by the inference service. The SARIMA models and the LSTM weights were trained once on this series and are kept unchanged by the DevOps study, which only deploys and operates them.

Variable	Count	Mean	Std	Min	Max
Rain (mm)	26,700	2.19	7.15	0.00	132.00
Temp Max (deg C)	26,828	30.14	3.00	19.60	38.93
Temp Min (deg C)	26,825	19.40	2.47	0.00	73.80
Humidity (%)	26,825	78.51	5.38	41.56	100.00

Table 4.1 Summary statistics of the daily data (26,835 rows)

Figure 4.1 shows the monthly series that the models are trained on, computed from the daily file: monthly rainfall total, mean temperature and mean humidity from 1951 to 2024, together with a ten-year moving mean. The strong seasonality (monsoon rainfall peak, a warm pre-monsoon period and a humid monsoon season) is what makes SARIMA with a period of twelve months appropriate, and the slow upward drift of humidity in the later decades is the cause of the drift alarms reported in Chapter 6.

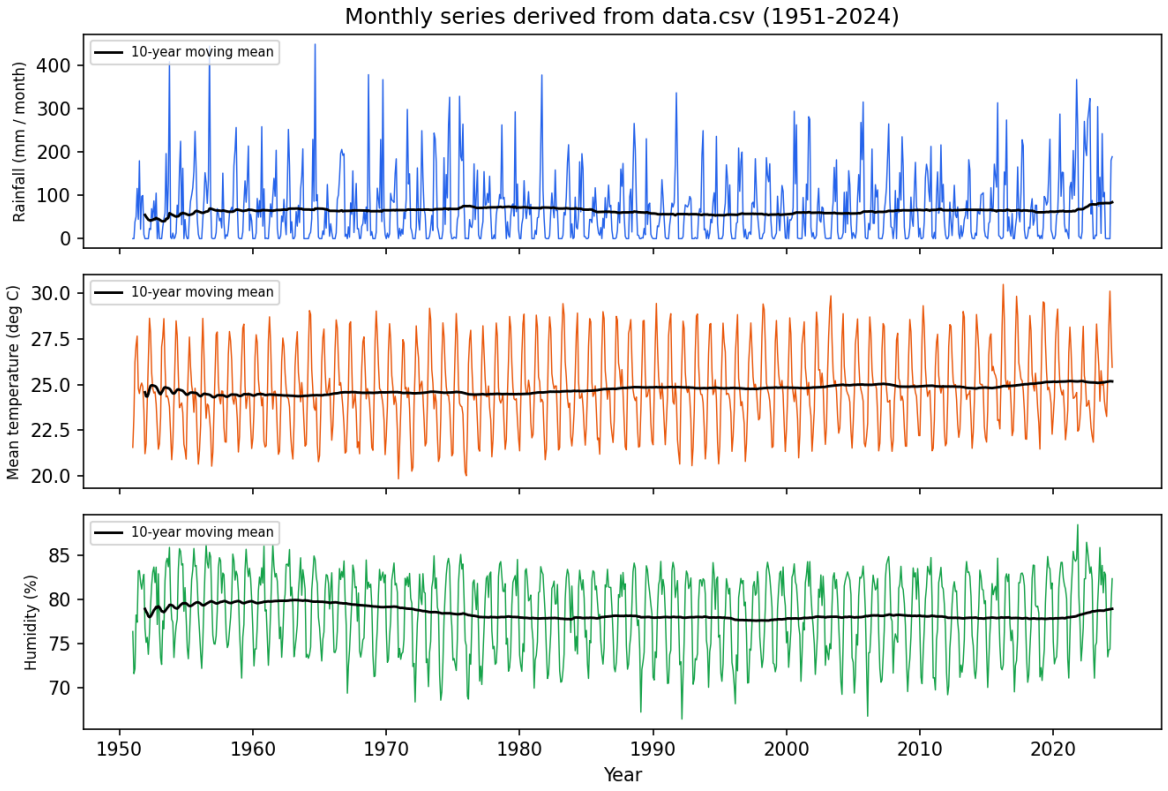


Figure 4.1 Monthly rainfall, temperature and humidity (1951-2024) from data.csv

The monthly climatology in Figure 4.2 (the average of each calendar month over all years) is the third tier of the degradation chain: it is always valid by construction, it needs no model, and it is used when both models are rejected or too slow.

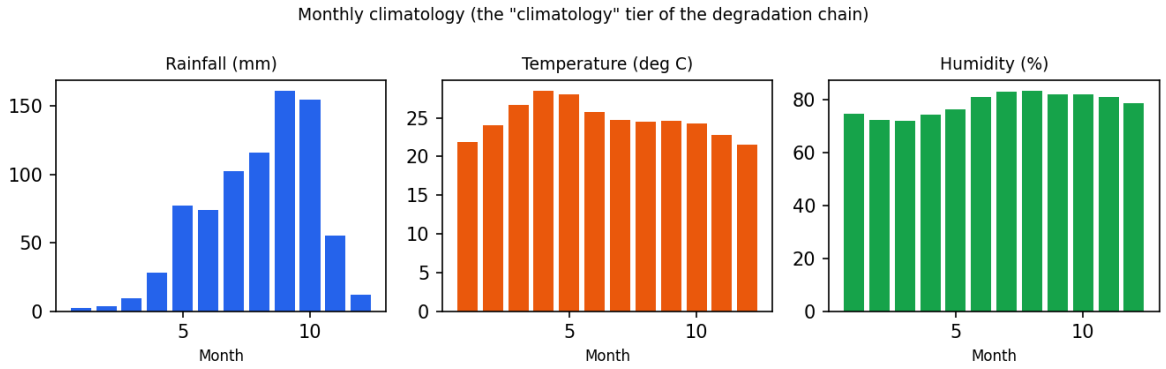


Figure 4.2 Monthly climatology used by the climatology tier

Calibration and evaluation of the guard respect time order: calibration windows come from years up to 2018 and hold-out windows from later years. The data contain a few obviously erroneous daily minima (a maximum of 73.8 degrees Celsius in Temp Min); these are retained because the monthly aggregation and the conformal calibration absorb them, and because removing them would change the original application's behaviour, which the project promises not to do.

4.3 Design and Methodology

The study follows an experimental-systems method: define baselines, define faults, define metrics, run repeated trials under identical conditions, and compare configurations with confidence intervals and exact tests. Figure 4.3 gives the overall flow.

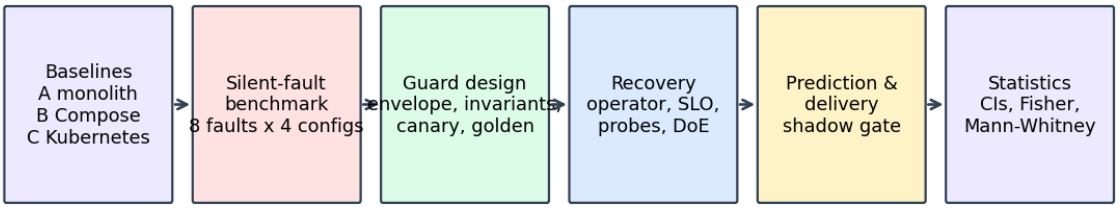


Figure 4.3 Methodology of the study

4.3.1 Configurations compared

Config	Description	Purpose
A	Original Flask monolith, SARIMA + LSTM in one process (SQLite)	Original application baseline
B	Docker Compose: frontend, inference API, PostgreSQL	Container baseline with restart policy
C	Kubernetes with default liveness/readiness probes	Default self-healing baseline
D	Kubernetes + semantic guard and tiered degradation	First correctness-aware version
C4 / D4 / E4	Image :4 with guard off / envelope / envelope + invariants	Controlled silent-fault benchmark
G4	E4 + golden-output integrity check (image :10)	Detects subtle corruption
H4	G4 + deadline- and load-aware degradation (image :12, 800 ms)	Handles slow tiers

Table 4.2 Configurations

4.3.2 Fault model

Fault	What is injected	Visible to default probes?
crash / hang / oom	Process exit, blocked requests, memory exhaustion	Yes (liveness/readiness fail)
corrupt	Output is NaN / garbage with HTTP 200	No
bitflip_severe	Exponent bits flipped in LSTM kernel weights, large output change	No
bitflip	Eight exponent most-significant-bit flips; output stays plausible	No
scaler	Input scaler parameters replaced (wrong units/scale)	No
unitchange	Temperature or rainfall unit changed in the output	No
stale	Old forecast served again instead of a fresh one	No
flatline	Output is constant over the horizon	No
delay	30 percent of model calls sleep 1.5 s	Partly (slow, not wrong)
leak / drift / dropvar	Memory growth, input distribution drift, missing input variable	Gradual

Table 4.3 Injected faults (SilentBench and extensions)

4.3.3 Metrics

- Share of answers that are correct, wrong (invalid values), fallback (valid answer from another model) or degraded (climatology or cache), and share of failed requests, measured by a client that knows the golden answer.
- Detection time and recovery time after injection or after healing; restoration time to full quality for operator experiments.
- Failure rate and p95 latency under bursty load and under the delay fault.
- False-alarm rate on hold-out windows and in a ten-minute healthy soak.
- Predictor lead time and false alarms per ten minutes.
- Shadow-gate decisions on good and bad candidate models.

4.3.4 Experimental protocol

Each experiment is a script that deploys a configuration, verifies the image and environment of the running pod (an early run was found invalid because an image tag had not been built, so verification was added), injects one fault for a fixed window at about 5 requests per second, records the client-side classification of every response, heals the fault and records the recovery. Trials are repeated 5 to 10 times per cell. Raw results are appended to JSON-lines files that are never rewritten, and invalid runs are kept under names that mark them superseded. Statistics (bootstrap 95 percent intervals, Fisher exact test, Mann-Whitney U test) are computed afterwards by analysis/phase14_stats.py.

4.4 System Architecture

Figure 4.4 shows the deployed system in the Kubernetes namespace climate and the monitoring namespace. The user's browser talks to the Flask frontend, which stores users and history in PostgreSQL and calls the inference API over HTTP. The inference API hosts the models, the guard, the golden check, the tier chain and the deadline logic. The ForecastService operator and the failure predictor observe the inference API; Prometheus scrapes every component every 5 seconds; Grafana and Jaeger present metrics and traces; chaos and probe drivers run outside or inside the cluster to inject faults and measure.

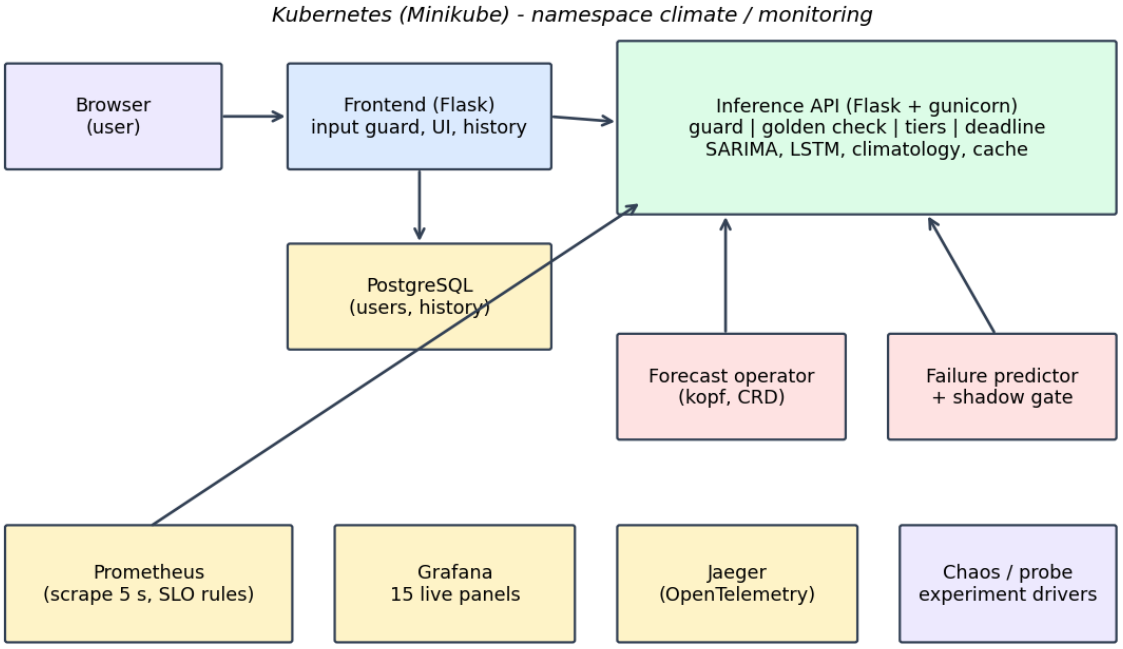


Figure 4.4 System architecture

Figure 4.5 shows the request path inside the inference API. A request starts at the requested tier (LSTM by default). The output of a tier is accepted only if it passes the envelope and invariant checks, the golden-output check and the deadline or queue condition; otherwise the tier is marked suspect (circuit breaker) or shed and the next tier is tried. A background canary every five seconds re-validates suspect tiers so that a repaired model returns to service without a restart.

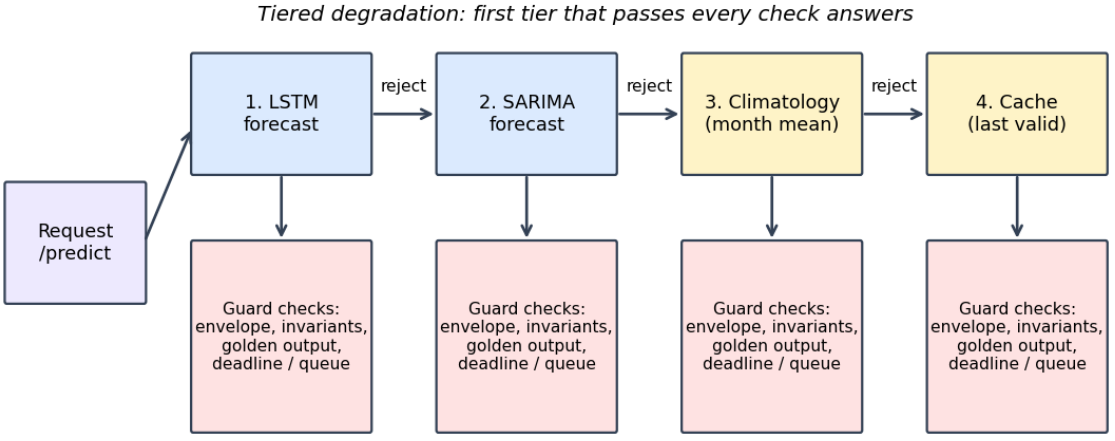


Figure 4.5 Tiered degradation and guard checks

4.4.1 Kubernetes objects

- Deployments: frontend, inference-api (gunicorn gthread, 1 worker, 16 threads),

postgres, jaeger, forecast-operator (scaled to zero except during operator experiments).

- Services: frontend (5000), inference-api (5001), postgres (5432), jaeger (16686, 4318).
- Configuration: ConfigMap for guard parameters and the policy table (forecast-policy); Secret for the (lab) database password and Flask secret key.
- CRD ForecastService with fields inferenceUrl, deployment, policy (rules or bandit), failureThreshold and cooldownSeconds.
- Probes: readiness on /health/ready or /health/semantic (variants), liveness on /health/live, startup probe for model loading.

4.5 Tools and APIs

Endpoint (inference API)	Purpose
GET /predict?model=&steps=&interval=	Forecast with tier fall-through; optional conformal intervals; returns model_used, tier, degraded
GET /health/live	Liveness: process answers
GET /health/ready[?partial=1]	Readiness: models loaded (partial mode accepts any usable tier)
GET /health/semantic	Per-model validity, score and golden deviation; semantic_ok flag
GET /metrics	Prometheus text metrics
GET /slo	Correctness SLI, Wilson lower bound, burn rates
GET /aging, /drift, /energy	Aging (Weibull), drift (PSI, Page-Hinkley) and energy estimates
GET /shadow/load, /status, /clear	Load a candidate LSTM, report shadow reject ratio, remove it
GET /golden/refresh	Recompute golden outputs after a validated promotion
GET /fault?mode=...	Inject or heal a fault (enabled only when FAULT_INJECTION=1)

Table 4.4 Inference API

Environment variable	Meaning	Default
SEMANTIC_GUARD	Enable the correctness guard	0
GUARD_MODE	envelope or envelope+inv	envelope
GOLDEN_CHECK / GOLDEN_TOL	Golden-output check and tolerance in history-std units	0 / 0.002
DEADLINE_MS	Per-request time budget (0 = off)	0
LSTM_CONC / SARIMA_CONC	Maximum concurrent calls per tier	2 / 3
ENERGY_AWARE	Order tiers by measured CPU cost	0
LOAD_MODE	eager or lazy LSTM loading	eager
SHADOW_LSTM_DIR	Directory of a candidate LSTM for shadow evaluation	-
OTEL_EXPORTER_OTLP_ENDPOINT	Enable OpenTelemetry tracing	-
FAULT_INJECTION	Enable /fault endpoints	0

Table 4.5 Configuration variables

Supporting tools: Python libraries (Flask, gunicorn, TensorFlow, statsmodels, scikit-learn, scipy, kopf, requests, matplotlib), Helm charts (Prometheus, Grafana, Kyverno, KEDA, Chaos Mesh), Argo Rollouts, cosign, Git and Git LFS, and a validation suite (validation/validate_all.py) with nineteen executed checks.

4.6 Summary

The design combines well-known building blocks (containers, probes, conformal

calibration, voting, SLOs, operators) into a pipeline that checks the answer instead of the process. The methodology compares seven configurations under ten fault types with repeated trials, and the architecture keeps the checks in the inference service, the recovery in an operator and the evidence in Prometheus, Grafana and Jaeger.

CHAPTER 5: IMPLEMENTATION AND TESTING

This chapter describes how the design was implemented, shows the most important pieces of code with an explanation of each, and documents the testing strategy and the defects that testing uncovered.

5.1 Implementation Requirements

- A reproducible build: every image is built from a Dockerfile with pinned Python dependencies (requirements.txt), tagged (inference :1 to :12, frontend :2 to :4, operator :1, :2) and loaded into Minikube with imagePullPolicy Never so that a tag always means the same bits.

- Verification before measuring: every experiment checks the image and environment of the running pod, because a silent build failure produced an invalid first run of configuration D.

- Idempotent experiment scripts with detached execution on Windows (PowerShell scripts started as hidden processes) so that long runs survive the closing of a terminal.

- Append-only result files and explicit naming of superseded or invalid data.

- Models kept exactly as trained: SARIMA pickles (about 118-124 MB per variable) and the LSTM .h5 files are stored with Git LFS and are never retrained by the DevOps code.

5.2 Implementation Tool Features

Tool	Feature used	Where
Flask + gunicorn (gthread)	Web framework; one worker with 16 threads so that shed or cached requests are served while a model call is running	frontend/app.py, inference-api/app.py, Dockerfile
TensorFlow / Keras	LSTM forecasting; tf.function wrapper reduced a forecast from 4.2 s to 0.15 s	inference-api/backends.py
statsmodels	SARIMA forecasting from fitted pickles	inference-api/backends.py
scikit-learn / scipy	Isolation Forest (predictor, input guard), KS test, Fisher exact, Mann-Whitney	analysis/
kopf	Python framework for Kubernetes operators; timer handler every 5 s	operator/forecast_operator.py
Prometheus client text format	Counters, gauges and histograms written by a small metrics module	inference-api/metrics.py
Helm	Install of Prometheus, Grafana, Kyverno, KEDA and Chaos Mesh	k8s/README.md
OpenTelemetry	Optional automatic tracing of Flask and requests with OTLP export to Jaeger	app.py (both services)
cosign + Kyverno	Image signing and admission-time signature verification	k8s/policy/

Tool	Feature used	Where
Argo Rollouts	Canary with Prometheus-based AnalysisTemplates	k8s/rollout/

Table 5.1 Implementation tools and the feature used

5.3 Code Snippets with Explanation

5.3.1 Window-level conformal calibration (guard.py)

The guard is calibrated on sliding windows. For every window length L used by the service, the family-wise score of every historical window in the calibration period is computed, and the empirical quantile at level 1 minus alpha (alpha = 0.01) is stored. A value-level bound would give a much higher window-level false-alarm rate (about 24 percent was measured with a per-value alpha before this correction), which is why the quantile is taken over the maximum score of a window.

```
    def calibrate_windows(self, cal_start=None, cal_end=None):
        """Family-wise (window-level) calibration: the quantile of the MAX score over sliding
windows of length L.
        A value-level 1-alpha bound over n values would otherwise give a much larger window-level
false-alarm rate."""
        m = self._monthly
        years = m['Year'].values.astype(int)
        months = m['Month'].values.astype(int)
        lo = cal_start if cal_start is not None else int(np.median(years))
        hi = cal_end if cal_end is not None else int(years.max())
        series = {v: m[v].values.astype(float) for v in VARS}
        env, inv = {L: [] for L in self.WIN_LENGTHS}, {L: [] for L in self.WIN_LENGTHS}
        n = len(years)
        for L in self.WIN_LENGTHS:
            for i in range(0, n - L):
                if not (lo <= years[i] <= hi):
                    continue
                w = {v: series[v][i:i + L] for v in VARS}
                env[L].append(self._env_raw(w, int(months[i])))
                if getattr(self, 'inv', None) and (self.inv['cross'] or self.inv['temporal']):
                    inv[L].append(self.invariant_score(w, int(months[i])))
        k = lambda arr: float(np.sort(arr)[min(len(arr) - 1, int(np.ceil((len(arr) + 1) * (1 -
self.alpha))) - 1)]) if len(arr) else 1.0
        self.qwin_env = {L: max(k(env[L]), 1e-9) for L in self.WIN_LENGTHS}
        self.qwin_inv = {L: max(k(inv[L]), 1.0) for L in self.WIN_LENGTHS}
        return self
```

Listing 5.1 Family-wise calibration of the envelope

5.3.2 The validity score (guard.py)

score() returns a single number that is at most 1 for a valid forecast. It normalises the envelope score and, in envelope+inv mode, the invariant score by the calibrated window quantiles and returns the larger of the two. A NaN output gives a non-finite score and is therefore always rejected.

```
    def score(self, fc, start_month=1):
        """Window-level normalised score: <= 1 valid. envelope (+ invariants if GUARD_MODE has
'inv')."""
        n = len(fc[VARS[0]])
        L = self._L(n)
        if not hasattr(self, 'qwin_env'):
            self.calibrate_windows()
        env = self._env_raw(fc, start_month) / self.qwin_env[L]
        if not np.isfinite(env) or 'inv' not in getattr(self, 'mode', 'envelope+inv'):
```

```
            return env
        return max(env, self.invariant_score(fc, start_month) / self.qwin_inv[L])
```

Listing 5.2 Window-level validity score

5.3.3 Golden-output integrity check (app.py)

At start-up the service stores a 24-step forecast for each model. golden_dev() returns the largest deviation of a later forecast from it, in units of the standard deviation of the history of each variable, and infinity if any value is not finite. Because the models are deterministic the healthy deviation is zero; the tolerance 0.002 only absorbs floating-point noise. The check catches the subtle bit-flip fault that stays inside the envelope.

```
def golden_dev(tier, fc):
    """Largest deviation from the healthy start-up forecast, in history-std units (inf if non-
finite). 0 if no golden output."""
    g = GOLDEN.get(tier)
    if not g:
        return 0.0
    k = min(len(fc[VARS[0]]), 24)
    dev = 0.0
    for v in VARS:
        x = np.asarray(fc[v][:k], dtype=float)
        if not np.all(np.isfinite(x)):
            return float('inf')
        dev = max(dev, float(np.max(np.abs(x - np.asarray(g[v][:k], dtype=float)))) / SD[v])
    return dev
FAULTS = os.environ.get('FAULT_INJECTION') == '1'
```

Listing 5.3 Golden-output deviation

5.3.4 Deadline- and load-aware tier call (app.py)

timed_run() is the only place where a model is called when DEADLINE_MS is set. It first tries to take a slot in the tier's bounded semaphore without waiting; if the queue is full the tier is shed with reason load. It then checks the remaining budget, submits the call to a worker pool and waits for at most the remaining time; a timeout sheds the tier with reason timeout. The semaphore is released by the worker itself, so a stuck call keeps occupying its slot and cannot be piled up by later requests.

```
def timed_run(b, steps, t_end):
    """Deadline- and load-aware call: None means 'shed this tier' (queue full or budget exhausted)
so the caller falls to a cheaper tier."""
    if DEADLINE_S <= 0:
        return run_backend(b, steps)
    sem = _SEM.get(b.name)
    if sem and not sem.acquire(blocking=False):
        metrics.inc('shed', tier=b.name, reason='load')
        return None
    remaining = t_end - time.time() - _RESERVE_S
    if remaining <= 0:
        if sem:
            sem.release()
        metrics.inc('shed', tier=b.name, reason='deadline')
        return None
    fut =  EXEC.submit( guarded, b, steps, sem)
    try:
        return fut.result(timeout=remaining)
    except Exception as e:
        if isinstance(e, TimeoutError) or type(e).__name__ == 'TimeoutError':
            metrics.inc('shed', tier=b.name, reason='timeout')
```

```
            return None
        raise



_FAULT_MODE = {'mode': None}
_SAVED = {}
FAULT_MODES = ('corrupt', 'bitflip', 'bitflip_severe', 'stale', 'scaler', 'unitchange',
'flatline', 'delay')
```

Listing 5.4 Deadline- and load-aware call

5.3.5 Tier fall-through with circuit breaker (app.py)

serve() walks the tier chain. A tier that is not ready or was marked suspect by the canary is skipped; a tier whose output fails the envelope, invariant or golden check is marked suspect (circuit breaker) and the next tier is tried; the climatology tier and the cache are the last resort. The first tier whose output passes every check answers and its forecast is cached.

```
    for tier in tier_chain(model):
        if tier in BACKENDS:
            b = BACKENDS[tier]
            if not b.ready or _SUSPECT.get(tier):      # canary marked this version suspect
(cross-model vote)
                continue
            fc = timed_run(b, steps, t_end)
            if fc is None:
                continue
            score = GUARD.score(fc, b.start_month)   # cheap conformal envelope check on the
output itself
            metrics.gauge('semantic_score', round(score, 3) if np.isfinite(score) else 1e9,
backend=tier)
            if score <= 1.0 and GOLDEN_ON and golden_dev(tier, fc) > GOLDEN_TOL:
                _SUSPECT[tier] = True
                metrics.inc('golden_reject', backend=tier)
                continue
            if score <= 1.0:
                _CACHE[steps] = fc
                return fc, tier, info
            _SUSPECT[tier] = True          # circuit breaker: skip this tier until the next canary
re-validates it
            metrics.inc('semantic_reject', backend=tier)
        elif tier == 'climatology' and not DISABLE_CLIM:
            ref = BACKENDS['lstm'].start_month if 'lstm' in BACKENDS else 1
            sm = BACKENDS[model].start_month if model in BACKENDS else ref
            return GUARD.climatology(steps, sm), 'climatology', info
        elif tier == 'cache' and steps in _CACHE:
            return _CACHE[steps], 'cache', info
    return None, None, info



def canary_once(steps=6):
    """Novelty 2 as a periodic canary: run every version, align calendar months, vote, update
suspect flags."""
```

Listing 5.5 Tier chain inside serve()

5.3.6 Operator decision and reconcile loop (forecast_operator.py)

The operator reads /health/semantic every five seconds, derives a signature (ok, unreachable, or invalid with the list of invalid models) and counts consecutive bad checks. When the threshold is reached and the cool-down has passed, choose() returns the action: the rules policy always restarts; the bandit policy looks the signature up in

the policy table and picks the action with the lowest expected cost. Restart is performed by patching the pod template annotation, which triggers a rolling update.

```
def signature(health):
    """Observable fault signature from /health/semantic (or connection failure)."""
    if health is None:
        return 'unreachable'
    if health.get('semantic_ok'):
        return 'ok'
    be = health.get('backends', {})
    bad = sorted(k for k, v in be.items() if not v.get('valid'))
    return 'invalid:' + ','.join(bad)


def choose(policy, sig, table):
    if sig == 'ok':
        return 'none'
    if policy == 'bandit' and sig in table:
        vals = table[sig]            # {twin action: expected cost}
        best = min(vals, key=vals.get)
        # twin action names -> operator actions (tier_switch is performed by the service itself,
no restart)
        return {'restart_only': 'restart', 'tier_switch': 'none', 'switch_then_restart':
'restart'}.get(best, 'restart')
    return 'restart'                 # rules policy / unseen signature
```

Listing 5.6 Signature and policy choice

5.3.7 Cost model of the digital twin (twin_bandit.py)

The first version of the cost ignored the fact that a tier switch leaves the service on a fallback tier forever, so tier_switch looked cheapest and the bandit never restarted (restored only 5 of 20 trials). The corrected cost adds W_Q seconds for every second spent below full quality, with a 240-second horizon when full quality never returns.

```
        full = HORIZON if (act == 'tier_switch' and r['fault'] in SILENT) else rec      # a tier
switch never restores full quality for silent faults
        cost = rec + W_FAIL * (r.get('failed_pct') or 0.0) + W_DEG * deg * 100 + W_Q * full
        eps[(r['fault'], act)].append(cost)
```

Listing 5.7 Quality term added to the cost

5.3.8 Container image of the inference service (Dockerfile)

The inference image starts gunicorn with one worker and sixteen threads (gthread). One worker keeps a single copy of the models in memory (a TensorFlow process needs about 1 GiB), and the thread count was raised from four to sixteen after a burst experiment showed that four threads were exhausted by queued requests, which had been mistaken for a CPU limit. The timeout of 120 seconds only protects against a stuck worker; request-level deadlines are handled in the application.

```
FROM python:3.10-slim
ENV PYTHONUNBUFFERED=1 MPLBACKEND=Agg
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5001
```

```
CMD ["gunicorn","-w","1","--worker-class","gthread","--threads","16","-b","0.0.0.0:5001","--
timeout","120","app:app"]
```

Listing 5.8 Dockerfile of the inference service

5.3.9 Unit test of the guard (test_guard.py)

The unit tests construct a synthetic monthly data set with a seasonal pattern and check the three basic behaviours of the guard: a NaN forecast and a grossly scaled forecast are rejected, and the climatology forecast is accepted. They run in under a second and are part of the validation suite.

```
def test_nan_and_outlier_rejected_clean_passes():
    g = SemanticGuard(synth())
    clean = g.climatology(6, 1)
    assert g.valid(clean, 1)
    assert not g.valid({v: np.full(6, np.nan) for v in VARS}, 1)
    assert not g.valid({v: clean[v] * 4 + 50 for v in VARS}, 1)
```

Listing 5.9 Guard unit test

5.3.10 Event-driven scaling object (KEDA)

The KEDA object scales the inference deployment between one and three replicas using a Prometheus query on the request rate. It was used in the scaling extra; on a single node it did not improve the burst outcome, which is reported as a limitation.

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata: {name: inference-api, namespace: climate}
spec:
  scaleTargetRef: {name: inference-api}
  minReplicaCount: 1
  maxReplicaCount: 3
  cooldownPeriod: 60
  pollingInterval: 5
  triggers:
  - type: prometheus
    metadata:
      serverAddress: http://prom-prometheus-server.monitoring.svc
      query: sum(rate(predict_seconds_count[30s]))
      threshold: "3"
```

Listing 5.10 KEDA ScaledObject

5.4 Testing

5.4.1 Test levels

- Unit tests (pytest): the guard rejects NaN and gross outliers and accepts clean and climatology forecasts; the invariant check rejects a flat-line.

- Integration checks inside the validation suite: service with real models in baseline and guard modes, cluster readiness and semantic health, fault cycle corrupt then heal inside

the pod, frontend pages through Kubernetes with PostgreSQL persistence, Prometheus and Grafana health.

- Data-integrity checks: row counts and class percentages of every result file (baseline experiments, SilentBench, operator trials, probe validation, predictor, shadow gate, extras, fix rounds).

- Experiments (Chapter 6): each experiment is also a test of a hypothesis; negative outcomes were recorded.

- Browser walk-through: every application page returned HTTP 200, plots and badges rendered, Grafana showed live panels and the new golden-check panels reacted to an injected fault, Jaeger showed traces that span the frontend and the inference API.

5.4.2 Validation suite

Check	Result	Evidence
V01 docker + kubectl available	PASS	29.8.2 / { "clientVersion": { "major": "1", "minor": "36",
V02 cluster pods Running	PASS	4 pods Running
V03 images present in minikube	PASS	climate-inference:4 and climate-frontend:2 present in minikube
V04 all python files compile	PASS	112 python files compile
V05 unit tests (pytest)	PASS	2 passed in 1.28s
V06 data and model files	PASS	daily rows=26835, sarima MB={'sarima_rainfall.pkl': 124, 'sarima_temp_mean.pkl': 124, 'sarima_humidity.pkl': 124}, 3 lstm .h5 present
V07 service, baseline mode (real models)	PASS	rain[0] sarima=5.0 lstm=87.295; metrics exposed
V08 service, guard on: faults detected + heal (real models)	PASS	detected -> tiers {'corrupt': 'climatology', 'unitchange': 'climatology', 'flatline': 'climatology', 'scaler': 'sarima'}; heal restores lstm identical
V09 guard ablation reproduces stored numbers	PASS	hold-out false alarm 0.0; detection unit=1.0 flatline=1.0
V10 cluster readiness + semantic health	PASS	ready=True; semantic_ok=True
V11 cluster fault cycle corrupt/heal (in pod)	PASS	guard=True: corrupt -> tier=climatology degraded=True; healed tier=lstm
V12 frontend flows through Kubernetes + PostgreSQL	PASS	pages {'healthz': 200, 'dashboard': 200, 'result': 200, 'lstm_result': 200, 'history': 200, 'analysis': 200}; user persisted in PostgreSQL
V13 Prometheus metrics + Grafana health	PASS	prometheus series=6 ; grafana ok
V14 baseline experiment data integrity	PASS	75 fault rows in 15 cells (5 each); 20 load rows
V15 silent-benchmark data integrity	PASS	332 rows, 41 cells; baseline healthy, class percentages sum to 100
V16 phases 7-13 result data (operator, probe validation, predictor, shadow gate, family)	PASS	operator rows=60 (rules restored 20/20) ; probe_validation rows=25 ; predictor: 6/6 leak and 6/6 drift failures analysed ; shadow gate: 15/15 bad reje
V17 cluster extras results (semantic probes, partial readiness, UI, scaling, Chronos)	PASS	extras rows=49; UI checks all True; chronos status=ok; scaling rows=9
V18 fix round: golden check + cost-aware bandit	PASS	G4 bitflip rows=20 (0 wrong), healthy rows=5 (100% correct); bandit v2 restored 20/20
V19 deadline-aware degradation (delay fault)	PASS	delay fault failed %: G4 mean 96.8 -> H4 0.0; H4 healthy 100% correct, 0% wrong on corrupt/bitflip/flatline
V20 metamorphic probe evaluation (experimental)	PASS	interleaved FA 2.0 %, time-split FA 66.0 % (limit), bitflip 93.3 % marginal, scaler 16.7 %
V21 baselines, HBW/MTTC and compound faults	PASS	baselines: simple detectors weak or noisy, golden strong but costly; compound bitflip+delay: E4 100 % wrong, H4 0 %; HBW C4 30 s vs G4 0 s

Table 5.2 Executed validation checks (19 of 19 passed)

5.4.3 Defects found by testing and how they were fixed

Defect	Root cause	Fix
Configuration D first run invalid	Image :3 was never built (silent build failure)	Pod image and environment verified before each run
Per-request cross-model vote crashed the pod	Voting on every request saturated the single worker	Background canary every 5 s; LSTM wrapped in tf.function (4.2 s to 0.15 s)

Defect	Root cause	Fix
About 24 percent window-level false alarms	Per-value alpha instead of window-level quantile	Family-wise window calibration
Isolation Forest could not extrapolate leak trend	Tree models do not extrapolate; warm-up rows distort training	Drop warm-up rows and add a Mahalanobis distance
Liveness timeouts under load	Synchronous gunicorn worker blocked the probe	gthread worker class with 4, later 16, threads
Subtle bit flips undetected by all checks	Corrupted output stayed inside the envelope	Golden-output integrity check (Section 6.1.2)
Bandit never restarted	Cost ignored time below full quality; policy table built before operator data existed	Quality term in the cost and regenerated table (Section 6.2.2)
Delay fault failed 96.8 percent of requests	Slow tier blocked all threads and the shared lock	Deadline- and load-aware degradation (Section 6.5)
Earlier scaling failures	Only 4 gunicorn threads, not CPU, limited the burst	Threads raised to 16; earlier claim withdrawn
Rollout stalled after kubectl set image with wildcard	The init container image was replaced as well	Containers named explicitly
Cluster overloaded by extra stacks	Kyverno, KEDA, Chaos Mesh and Argo running together with the lab	Uninstalled after their experiments

Table 5.3 Defects found and fixed

5.5 Summary

The implementation consists of a guarded inference service, a Flask frontend, a kopf operator and a large set of experiment and analysis scripts. Testing combined unit tests, a nineteen-check validation suite, data-integrity checks and a browser walk-through, and it uncovered eleven defects that were fixed from their root causes.

CHAPTER 6: RESULTS AND ANALYSIS

This chapter reports the measured results. All numbers come from executed experiments whose raw files are stored in the project (results folder); sample sizes are stated for every table because they are small (5 to 10 trials per cell, one laptop). Where a result is negative or inconclusive it is reported as such.

6.1 Results

6.1.1 Baselines: monolith, Docker Compose and default Kubernetes

The first experiment set injects infrastructure faults (crash, hang, out-of-memory) and the simplest silent fault (corrupt) into configurations B (Compose) and C (Kubernetes) and, for a defined list, D (guarded Kubernetes). Table 6.1 gives the mean recovery time and the mean share of failed requests. Crash recovery is dominated by the container restart delay (about 28 s of a 34.6 s total; application start-up is 4.6 s). For the corrupt fault the default configurations never recover: the pod stays Running and Ready while every answer is wrong.

Config	Fault	Trials	Recovered	Mean recovery (s)	Mean failed requests (%)
A	crash	5	5/5	2.8	54.1
A	hang	5	0/5	-	0.0
A	oom	5	0/5	-	0.0
B	corrupt	5	0/5	-	100.0
B	crash	5	5/5	47.4	97.7
B	hang	5	2/5	183.4	99.8
B	oom	5	5/5	32.8	96.9
C	corrupt	5	0/5	-	100.0
C	crash	5	5/5	15.6	88.9
C	hang	5	5/5	61.9	97.6
C	oom	5	5/5	13.1	87.1
D	corrupt	5	4/5	3.5	15.7
D	crash	5	5/5	19.5	90.6
D	hang	5	5/5	77.1	98.1
D	oom	5	5/5	43.3	95.1

Table 6.1 Baseline fault trials (mean over trials; recovery - means never recovered)

6.1.2 Silent-fault benchmark (SilentBench)

Table 6.2 is the central result of the study. For eight faults and four configurations it shows the share of answers that were wrong during a 30-second fault window at 5 requests per second, as a mean with a bootstrap 95 percent confidence interval and the number of trials; the last column is the p-value of Fisher's exact test comparing the guard-off configuration (C4) with the full guard (G4) on the trial-level outcome any

wrong answer. With the guard off, every silent fault produces 100 percent wrong answers, and the platform never detects one of them. With envelope and invariants (E4) all faults except the subtle bit flip are turned into valid answers (degraded climatology or fallback to SARIMA); the flat-line fault is missed by the envelope alone (D4) and caught only when the invariants are added. The subtle eight-flip bit fault is missed by C4, D4 and E4, and caught in 10 of 10 trials by G4 thanks to the golden-output check. The delay fault does not produce wrong answers but failed requests; it is handled in Section 6.5.

Fault	C4 guard off	D4 envelope	E4 + invariants	G4 + golden	Fisher C4 vs G4
corrupt	100.0 [100.0, 100.0] n=10	0.0 [0.0, 0.0] n=10	0.0 [0.0, 0.0] n=10	0.0 [0.0, 0.0] n=5	0.00033
unitchange	100.0 [100.0, 100.0] n=10	0.0 [0.0, 0.0] n=10	0.0 [0.0, 0.0] n=9	0.0 [0.0, 0.0] n=5	0.00033
flatline	100.0 [100.0, 100.0] n=10	99.93 [99.79, 100.0] n=10	0.0 [0.0, 0.0] n=10	0.0 [0.0, 0.0] n=5	0.00033
scaler	100.0 [100.0, 100.0] n=10	0.0 [0.0, 0.0] n=10	0.0 [0.0, 0.0] n=10	0.0 [0.0, 0.0] n=5	0.00033
stale	100.0 [100.0, 100.0] n=10	0.0 [0.0, 0.0] n=10	0.0 [0.0, 0.0] n=10	0.0 [0.0, 0.0] n=5	0.00033
bitflip	100.0 [100.0, 100.0] n=10	100.0 [100.0, 100.0] n=10	100.0 [100.0, 100.0] n=10	0.0 [0.0, 0.0] n=10	1.1e-05
bitflip_severe	100.0 [100.0, 100.0] n=10	0.0 [0.0, 0.0] n=10	0.0 [0.0, 0.0] n=10	0.0 [0.0, 0.0] n=10	1.1e-05
delay	0.0 [0.0, 0.0] n=10	0.0 [0.0, 0.0] n=10	0.0 [0.0, 0.0] n=10	0.0 [0.0, 0.0] n=10	1

Table 6.2 Wrong answers served (percent of responses in the fault window), mean [95 % CI], n trials

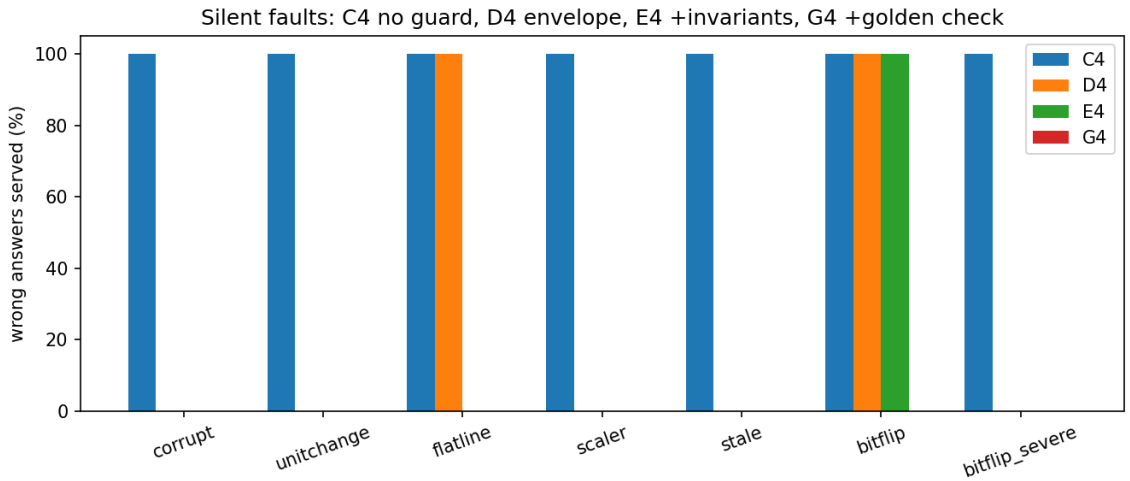


Figure 6.1 Wrong answers served per fault and configuration

In the healthy control (fault none) G4 and H4 served 100 percent correct answers in all trials, so no false alarm occurred; a ten-minute soak of the guarded service was also fully correct, and the offline hold-out evaluation (windows after 2018, alpha = 0.01, L = 6) gave a false-alarm rate of 0.0 percent for both the envelope and the envelope with invariants (with alpha = 0.05 it would have been 20 to 36.7 percent, which is why the lower alpha was chosen).

6.1.3 Guard ablation (offline)

Offline detection rates on hold-out windows (6-month windows): unit change 100 percent, scaler 100 percent, swapped variables 100 percent, NaN 100 percent, humidity minus 20 percent 80 percent, rainfall times three 73 percent, temperature plus 4 degrees 32 percent and flat-line 37 percent with the envelope alone and 100 percent with the invariants. The weak cases show the limit of a statistical envelope: a modest bias that stays within the climatological range is hard to see, which motivated the cross-model canary and the golden-output check. A comparison of static, 20-year rolling and adaptive conformal calibration (Gibbs-Candes style) showed the same false-alarm rate (0.0) and similar detection, so the simple static calibration was kept.

6.2 Benchmarking and Analysis

6.2.1 Layer complementarity

Fault	Envelope	+ Invariants	+ Golden output	Notes
corrupt (NaN)	caught	caught	caught	non-finite score
unitchange, scaler, stale	caught	caught	caught	gross change of range
bitflip_severe	caught	caught	caught	large output change
flatline	missed	caught	caught	constant series inside the range
bitflip (8 flips)	missed	missed	caught	plausible output, deviates from golden

Table 6.3 Which guard layer catches which fault (10 trials per cell)

The table shows that, among the per-request checks, each added layer closes a class of faults that the previous layers cannot see. Section 6.7 qualifies this: the golden check alone detects every fault but is too expensive to run per request. The cross-model canary is the fourth layer; it allows a repaired tier to return to service without a restart and detects a model that disagrees with the others even when its own output looks plausible.

6.2.2 Operator and recovery policies

Sixty operator trials were run on configuration E4: three policies (none, rules, bandit), four silent faults (corrupt, unitchange, scaler, flatline) and five trials each, with the fault left in place and the time to full quality measured. Without an operator the guard keeps serving valid degraded answers but full quality never returns (0 of 20). The rules policy restores 20 of 20 trials in 23.6 seconds on average (95 percent CI 22.3 to 24.9). The first version of the learned bandit policy restored only 5 of 20 (only the scaler fault), because its cost ignored the time spent below full quality and the policy table had been built before the operator data existed; after adding a quality term to the cost and regenerating the table it restored 20 of 20 in 21.6 seconds on average (CI 20.6 to 22.6). The 2-second difference to the rules policy is statistically detectable (Mann-Whitney $p = 0.041$) but

both policies choose the same action, so it is run-to-run drift and not a policy effect. The rules policy versus no operator is significant (Fisher exact p = 1.5e-11).

Policy	Restored to full quality	Mean time to full quality [95 % CI] (s)
none (operator off)	0 / 20	never
rules (restart after 3 invalid checks)	20 / 20	23.6 [22.3, 24.9]
bandit, version 1 (cost without quality term)	5 / 20	18.8 - 25.2 (scaler only)
bandit, version 2 (cost-aware)	20 / 20	21.6 [20.6, 22.6]

Table 6.4 Operator policies (60 + 20 trials)



Figure 6.2 Time to full quality under the operator

A second recovery observation concerns probe design. Making the readiness probe semantic and keeping only readiness leaves the service failed (92 percent failed requests, never restored); semantic readiness together with a liveness restart restored in 36 seconds; and a partial-readiness mode that accepts any usable tier kept serving with 0 percent failed requests during the fault, against 81 percent for strict readiness. Kubernetes self-healing thus works on correctness only if the probe is made to test correctness, and the choice between strict and partial readiness is a trade-off between never serving degraded answers and staying available.

6.2.3 Probe tuning, design of experiments and Bayesian optimisation

A 30-run central-composite design on the hang fault (probe period, failure threshold, replicas) was analysed with a quadratic response surface (leave-one-out R2 0.895 for recovery and 0.892 for failed share). The result is dominated by one factor: with one replica recovery takes about 52 seconds and 97 percent of requests fail; with three

replicas recovery is 1.3 to 3.3 seconds because other replicas keep answering. A bounded Bayesian optimisation recommended period 7, threshold 5 and three replicas; validation on held-out runs (Table 6.5) confirmed that the benefit comes from the replicas and not from the tuned probe parameters: at one replica the recommended probes were not better than the default (73 s versus 66 s).

Setting (period s / threshold / replicas)	Trials	Recovered	Median recovery (s)	Mean failed (%)	p vs default
default 10 / 3 / 1	5	5/5	66.3	97.5	-
default probes, 3 replicas	5	5/5	1.7	24.5	0.008
recommended 7 / 5 / 1	5	5/5	73.0	98.0	0.012 (worse)
recommended 7 / 5 / 3	5	5/5	1.7	25.2	0.012

Table 6.5 Validation of tuned probe settings (hang fault) - a negative result for probe tuning

6.2.4 Correctness SLO replay

Fault injection was replayed against the SLO rules (99 percent objective, 30-second and 2-minute windows, 5 requests per second). For the faults that corrupt values (corrupt, unit change) with the guard off the correctness alert fired in 3 of 3 trials after a median 9.2 and 15.5 seconds while 376-382 invalid answers were served in the 75-second window; for the flat-line and subtle bit-flip faults the validity check used by the SLO itself did not see the fault, so no alert fired - which is consistent with the guard ablation and is the reason the golden check was added. With the guard on, invalid answers served were zero and the degraded-answer alert fired instead.

6.2.5 Failure prediction

Telemetry windows from healthy runs were used to train an Isolation Forest and a Mahalanobis distance; leak and drift faults were injected six times each. For the memory-leak fault both detectors alarmed before the failure in 6 of 6 trials with a median lead time of 69.6 seconds, against 23.9 seconds for a static 80 percent memory threshold and no lead for the reactive guard; for the drift fault they alarmed in 6 of 6 trials with a median lead of about 23.7 seconds. On healthy data the Isolation Forest raised 2.45 false alarms per 10 minutes, which is the price of the lead time and is reported as a limitation.

6.2.6 Shadow-traffic gate and delivery

Candidate LSTM models were loaded into the live service in shadow mode while 5 requests per second were mirrored. The gate promotes a candidate only if its reject ratio

is at most 5 percent over at least 20 samples. All 15 bad candidates (wrong scaler, swapped variables and others) were rejected (reject ratio 1.00, median score 8.6-10.2) within about 8 seconds and all 6 good candidates (exact copy, small noise) were promoted (reject ratio 0.00, score 0.62-0.64) after about 13 seconds; a re-check gave 7 of 7 correct decisions. An Argo Rollouts canary with a Prometheus AnalysisTemplate promoted a good revision and aborted a revision whose gate was deliberately made to fail, which shows that the same criterion can drive a standard delivery tool.

Candidate	Expected	Decisions	Median reject ratio	Median score	Median s to decision
good_copy	promote	promote 3/3	0.00	0.64	12.9
good_noise	promote	promote 3/3	0.00	0.62	12.6
bad_scaler	reject	reject 3/3	1.00	8.64	8.2
bad_swap	reject	reject 3/3	1.00	10.21	8.2

Table 6.6 Shadow gate decisions (selected candidates; 15 of 15 bad rejected, 6 of 6 good promoted overall)

6.2.7 Extras, including negative results

Extra	Outcome
Aging and drift monitoring	Weibull shapes 1.03-1.34; PSI Humidity 3.07, Rainfall 0.61, Temp 0.35; Page-Hinkley alerts on all variables; model age 78 months with retraining recommended
Energy-aware tiers	Estimated energy per request 3.99 J (always LSTM) versus 0.93 J (SARIMA first) at 100 percent correct answers; p95 214 ms versus 101 ms
OpenTelemetry + Jaeger	One trace spans frontend GET /dashboard (1506 ms) and inference GET /predict (30 ms); 3 of 8 traces span both services
Kyverno + cosign	Policy rejected nginx:latest and unapproved images; signed image admitted; re-pushed or unsigned image denied (no signatures found)
Chaos Mesh	Pod kill: ready again after 16.1-16.3 s; 300 ms network delay raised p50 from 39 to 646 ms with no failures, back to 42 ms after removal
KEDA (request-rate trigger)	Scaled to 1-3 replicas; no benefit shown on a single node; earlier burst failures were traced to thread-pool exhaustion, not CPU
RL autoscaler (simulation only)	Tabular Q-learning lowest drop rate (2.4 percent) but reactive rule had the best reward (-110 versus -121); not validated on a cluster
Log anomaly detection	Zero log templates; zero of six silent faults detected from logs - logs carry no signal here
Probe tuning	No benefit beyond replicas (Table 6.5)
LSTM autoencoder / input guard	Many healthy false alarms (input guard 6 percent) - kept as weak results

Table 6.7 Additional components and their measured outcome

6.3 Screenshots and Live Plots

All screenshots in this section were taken from the running system in the built-in browser on 17 October 2026, after generating real traffic against the deployed service (more than 5,700 predictions served, including an injected bit-flip fault followed by healing). They show the application pages that were served by Kubernetes from PostgreSQL-backed sessions, the live Grafana dashboard with its 5-second refresh, the Prometheus expression browser and the Jaeger trace viewer. No image in this report is a mock-up.

6.3.1 Application pages



Figure 6.3 Home page of the Bangalore Climate Analytics System (browser, localhost:5000)

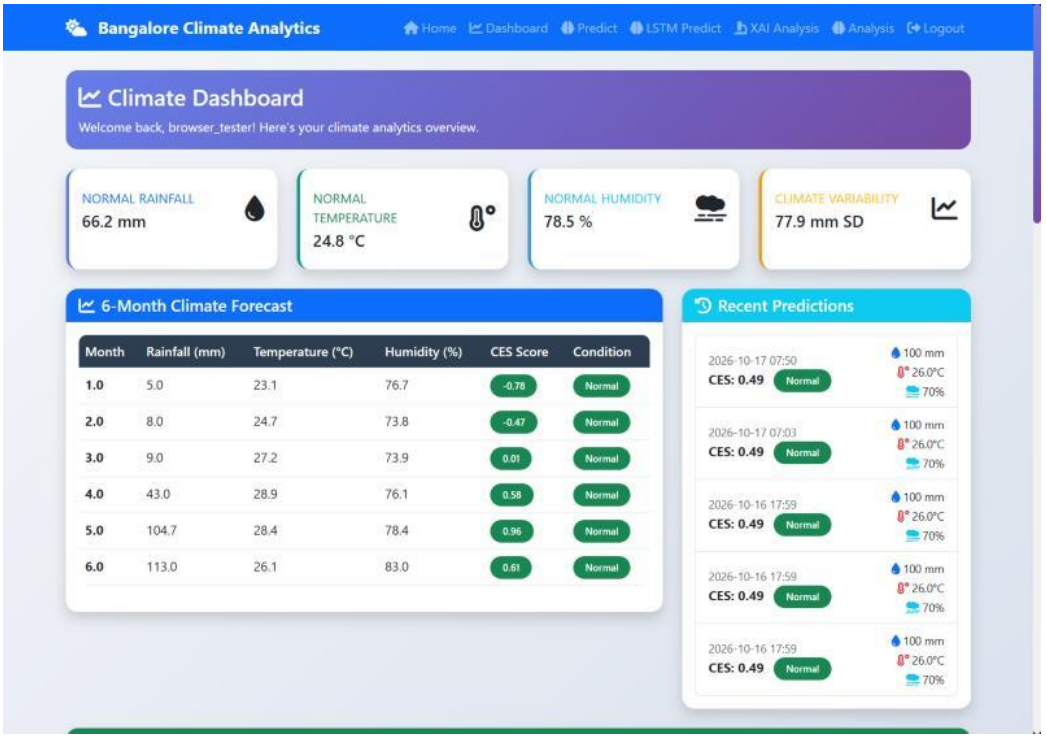


Figure 6.4 Climate dashboard: current values, 6-month forecast table and recent predictions

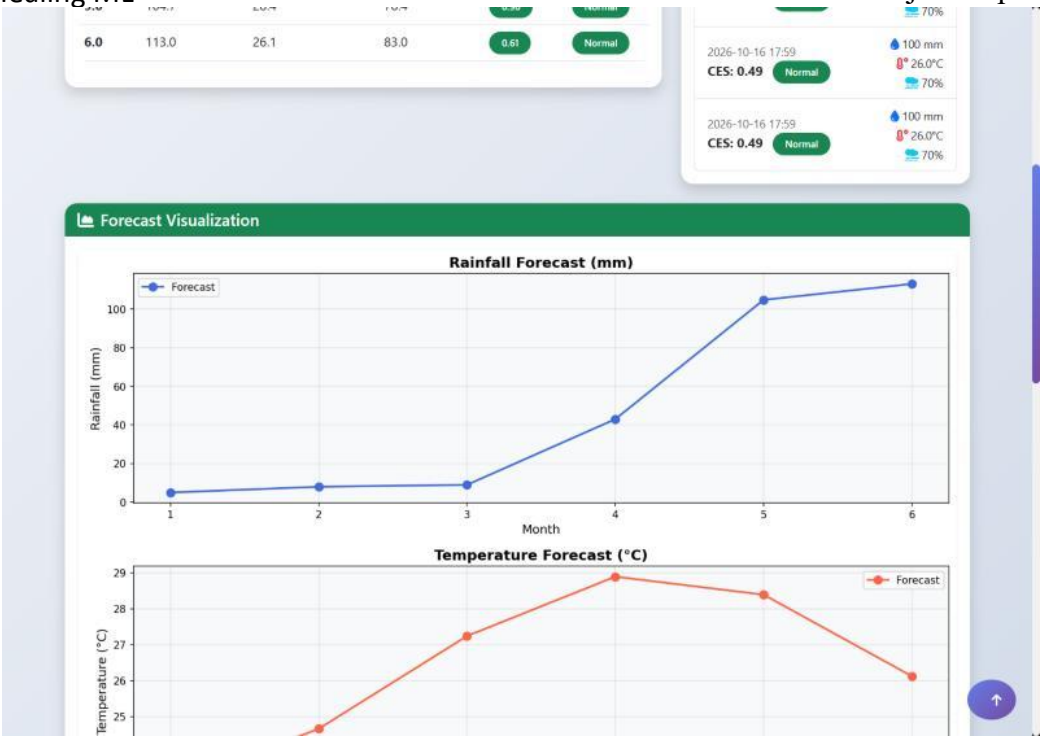


Figure 6.5 Dashboard forecast plots (rainfall and temperature)

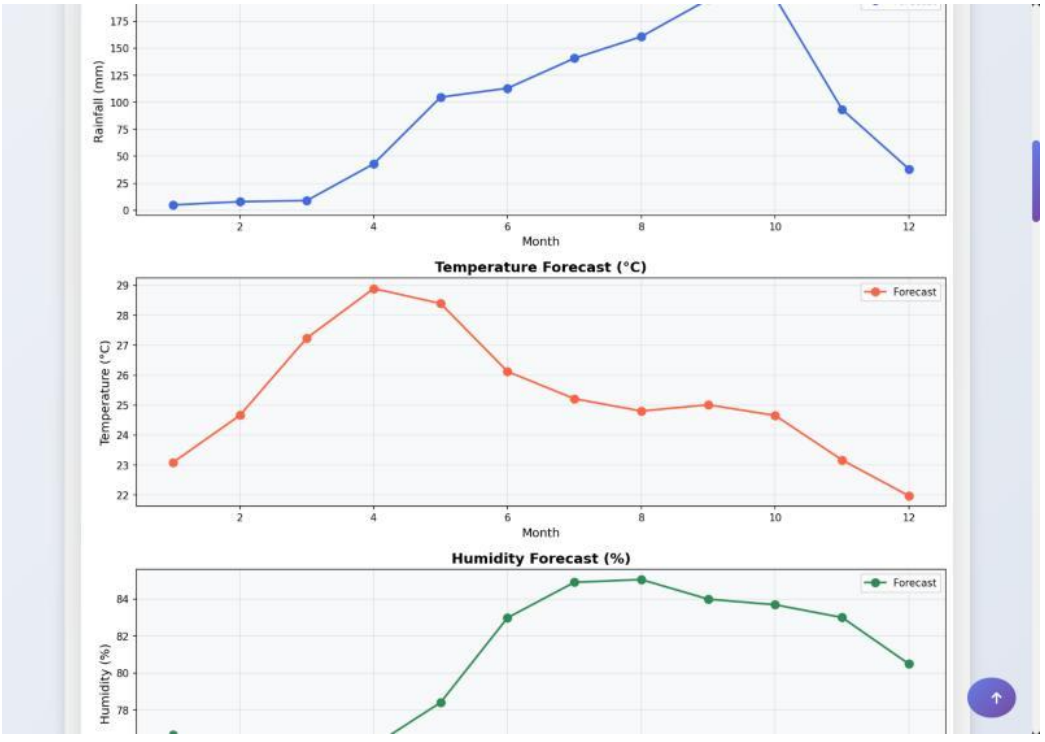


Figure 6.6 Twelve-month SARIMA forecast plots served by the inference tier



Figure 6.7 Analysis gallery: distributions, anomaly z-scores, anomaly evolution and decade-wise anomalies

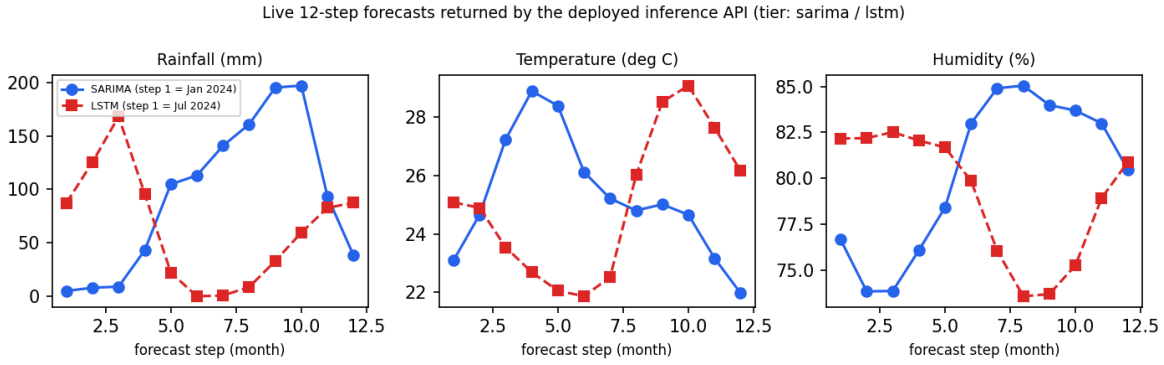


Figure 6.8 Live 12-step forecasts of SARIMA and LSTM returned by the deployed API

6.3.2 Live monitoring: Grafana and Prometheus

Figure 6.9 shows the first panels of the live Grafana dashboard: inference p95 latency per model, inference requests per second per model, frontend request rate by status, frontend p95 latency, model load time and model-ready status. The latency of the LSTM (yellow) is visibly higher than that of SARIMA (blue), and the request-rate panel shows the traffic that was generated for this screenshot. Figure 6.10 shows the golden-output panels after the bit flip was injected; Figures 6.11 and 6.12 show the same counters directly in the Prometheus expression browser: the golden violation counter for the LSTM jumps when the fault is injected, and the per-tier valid-answer counters show that the service kept answering from the LSTM, the climatology tier and (during the

fault) the fallback tiers.



Figure 6.9 Live Grafana dashboard, last 30 minutes (refresh 5 s)

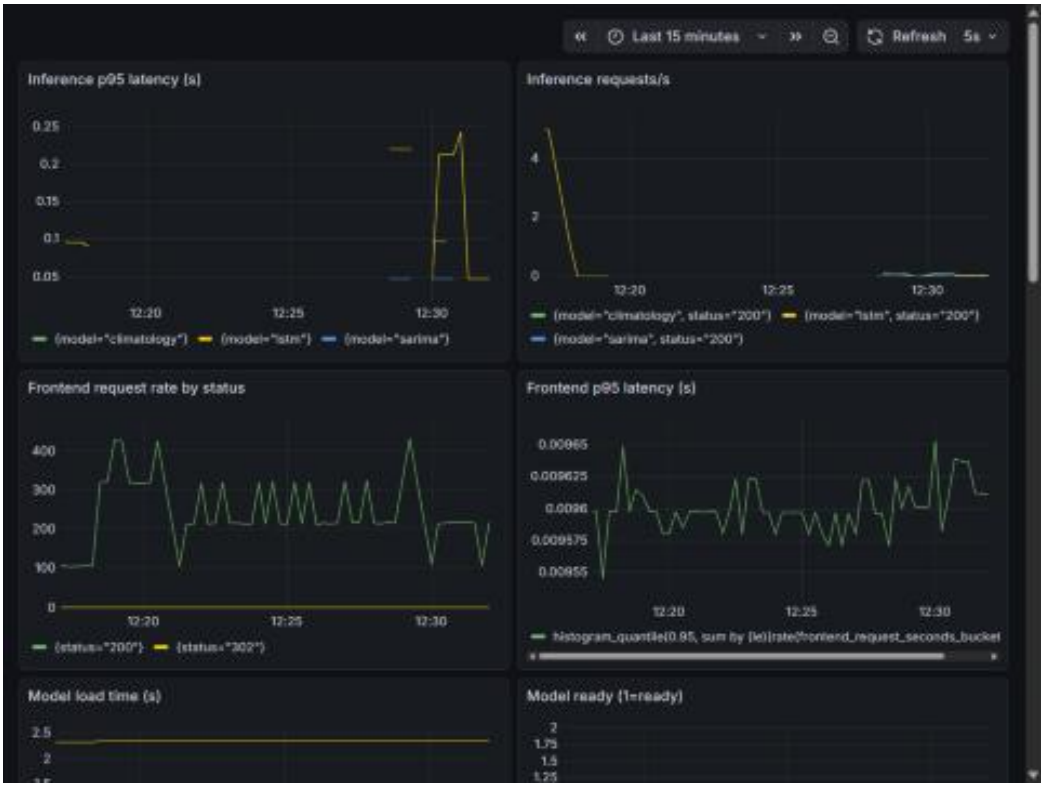


Figure 6.10 Grafana dashboard panels during the verification run

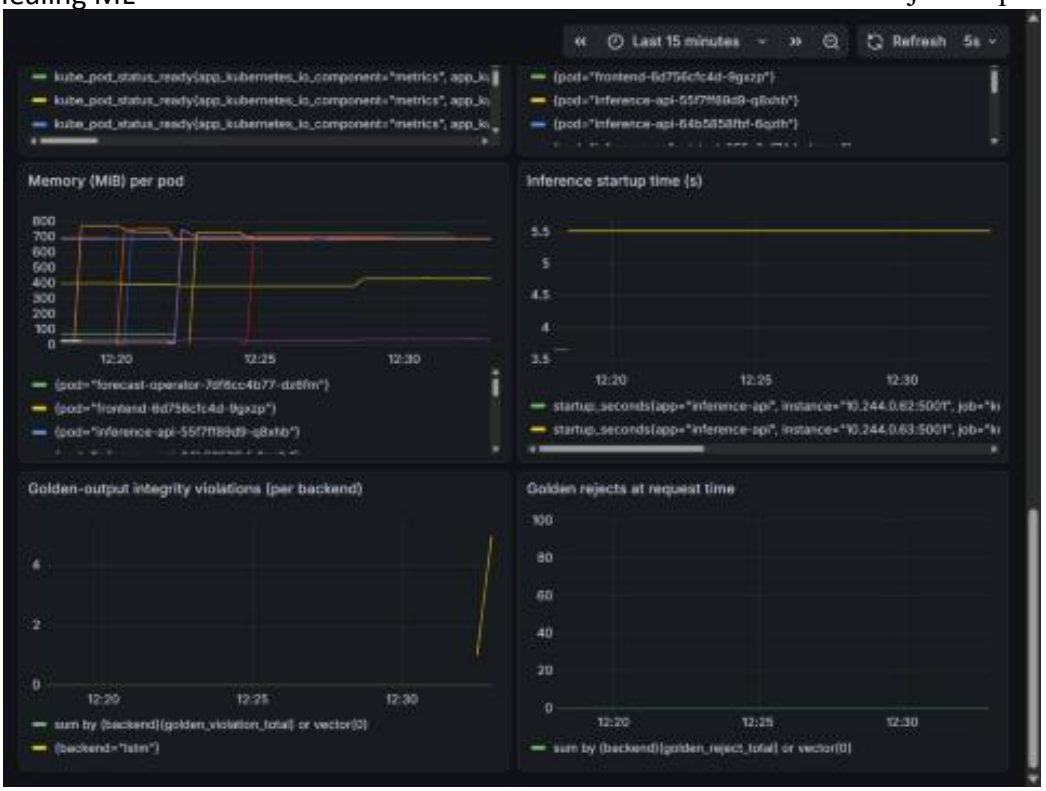


Figure 6.11 Golden-output integrity violations after an injected bit flip (Grafana panel)

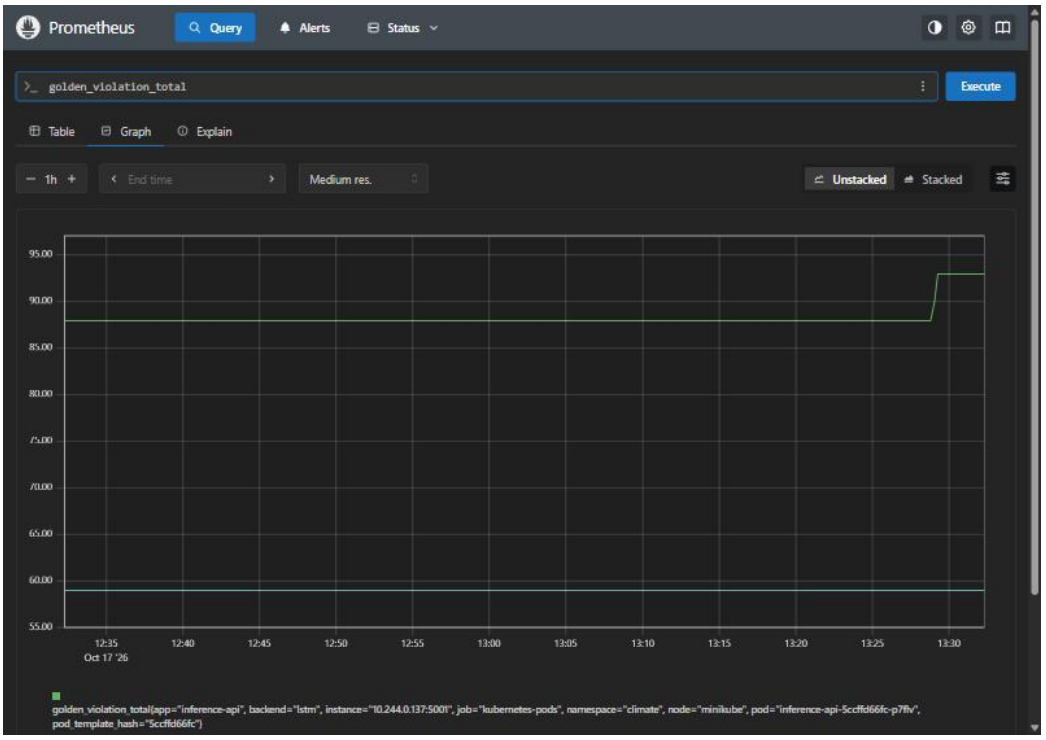


Figure 6.12 Prometheus: golden_violation_total for the LSTM backend (1 h)

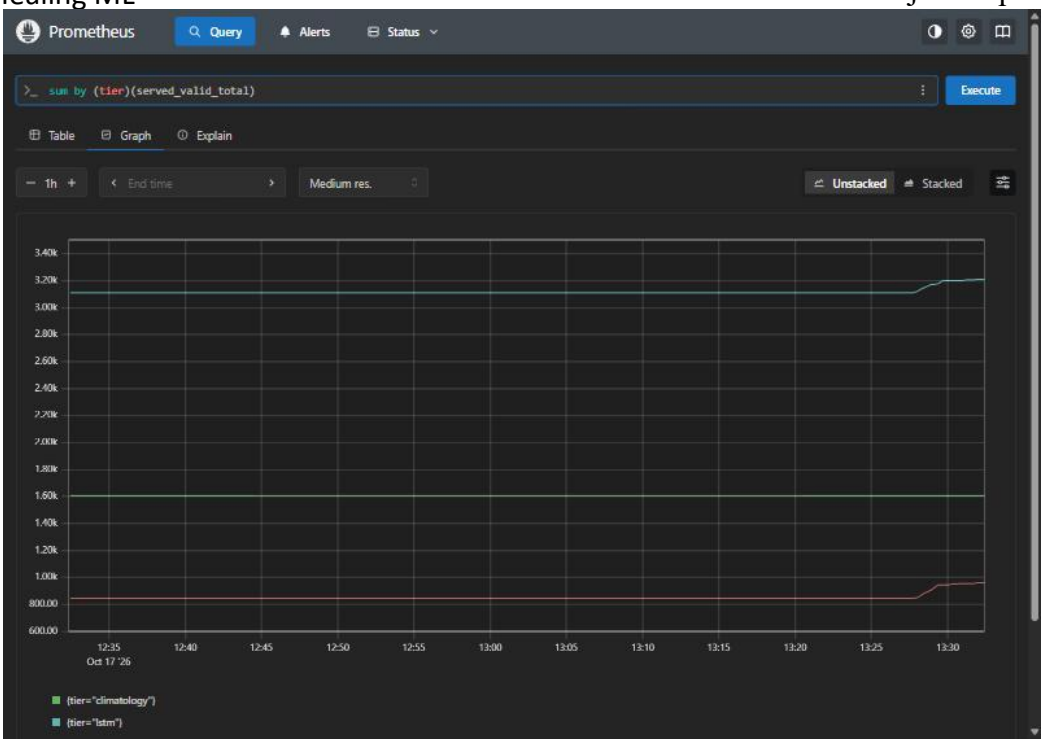


Figure 6.13 Prometheus: valid answers served per tier (1 h)

6.3.3 Distributed tracing

Figure 6.14 is the Jaeger view of one real request: the frontend handles GET /dashboard (1.81 s in total), and within the trace it calls the inference API three times (GET /predict, about 40 ms each), so the time that the user waits is dominated by the frontend work (database access and plotting) and not by the model service. Figure 6.15 is the trace search that lists traces crossing both services.

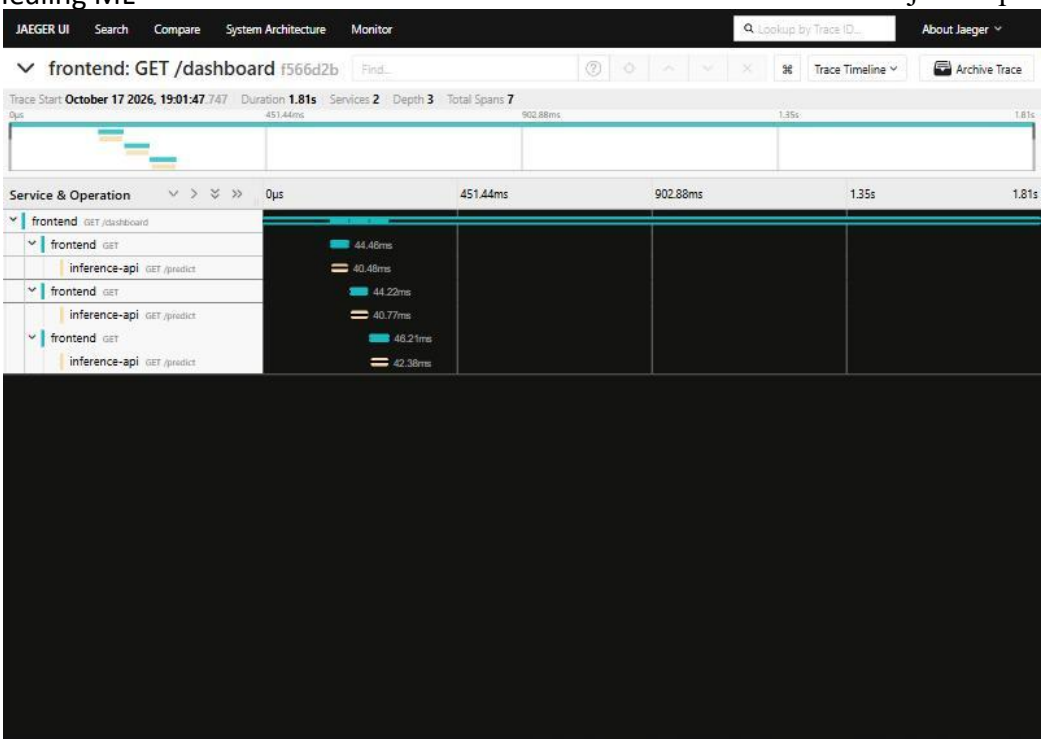


Figure 6.14 Jaeger trace of GET /dashboard with three inference calls

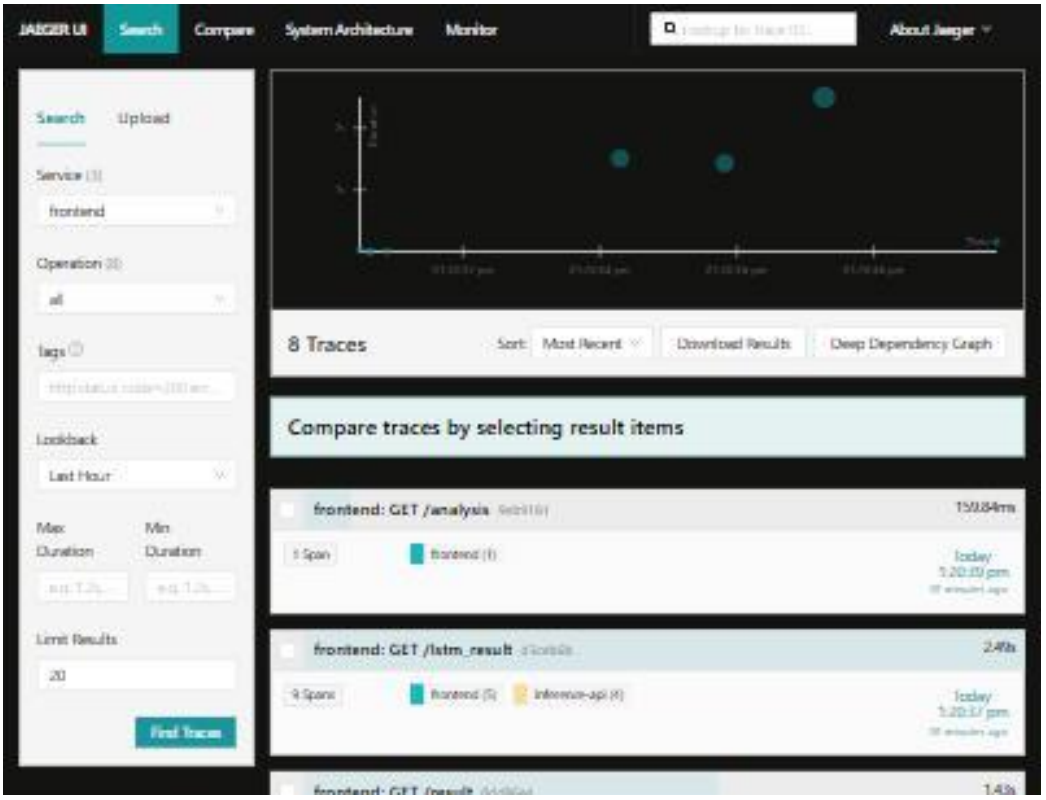


Figure 6.15 Jaeger trace search for the frontend service

6.4 Contributions (gap-based)

Six contributions are put forward. Several of their ingredients are known in the literature (Section 2.3); the contribution is the measured, integrated study that fills the gaps listed

there. Claims of priority are restricted to the reviewed literature and must be confirmed by a structured IEEE and Scholar search.

#	Component	What is new	Measured evidence
1	Silent-failure characterisation and SilentBench	First measured comparison of monolith, Compose and Kubernetes under faults that keep the service healthy but wrong	100 % wrong answers and 0 detections with default probes (10 trials per cell)
2	Layered correctness guard	Window-level conformal envelope + temporal invariants + cross-model canary + golden-output check applied as a health check of a Kubernetes forecasting service	0 % wrong answers for all eight faults with G4; 0 false alarms in healthy control and soak
3	Tiered degradation and correctness SLO	Four-tier fall-through with circuit breaker; SLO on served validity with Wilson bound and burn rates	All detectable faults become valid labelled answers; alerts after 9-15 s when values are corrupted
4	Correctness-aware self-healing	Semantic probes plus operator; cost model that charges time below full quality	Restoration 20/20 in about 22-24 s versus never; partial readiness 0 % vs 81 % failed
5	Failure prediction and model-aware delivery	Precursor predictor on model-serving telemetry; shadow gate on output validity	Leak lead 69.6 s (6/6); gate 15/15 bad rejected, 6/6 good promoted
6	Deadline- and load-aware degradation	Per-request budget and per-tier concurrency caps turn slow tiers into labelled lower-quality answers	Delay fault failed requests 96.8 % to 0 %; burst p95 about 4x lower

Table 6.8 Innovative components

6.5 Deadline- and Load-Aware Degradation (Novelty 6)

The delay fault makes 30 percent of the model calls sleep for 1.5 seconds. Without a deadline (G4) the sleeping calls occupy the service's threads and the shared model lock, requests queue behind them and 96.8 percent of requests fail on the client side with a p95 latency of about 2 seconds. With the deadline- and load-aware mechanism (H4: 800 ms budget, concurrency caps of two LSTM and three SARIMA calls) 0 percent of the requests failed, 94 percent were answered with valid climatology answers labelled as degraded, and the p95 latency fell to 762 ms; wrong answers were zero in both configurations. Most requests end on climatology because all backends still share one lock; per-tier locks could keep more of them on SARIMA and are listed as future work.

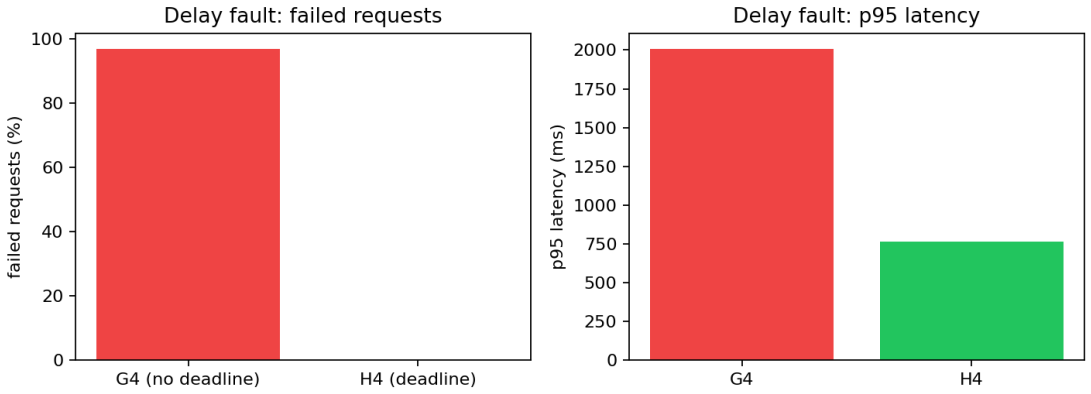


Figure 6.16 Delay fault with and without deadline-aware degradation (10 trials each)

Under a bursty LSTM load (3, then 20, then 3 requests per second for 40, 70 and 40 seconds on one replica) no requests failed in either configuration on the 16-thread image, but the p95 latency during the burst was 504, 1283 and 867 ms without the deadline and 185, 282 and 207 ms with it. The healthy control and the corrupt, bit-flip and flat-line faults behaved exactly as with G4 (100 percent correct, 100 percent degraded, 100 percent fallback and 100 percent degraded respectively, with no wrong answers). A byproduct of this experiment was a correction of an earlier conclusion: the high failure rates seen in the earlier scaling experiments were mostly caused by the use of four server threads, and the claim that scaling out does not help on a single node was withdrawn until it is re-measured.

6.6 Metamorphic Readiness Probe (experimental)

To obtain a correctness check that needs no stored reference output, a metamorphic probe was built. A metamorphic relation links the outputs of the model on two related inputs. Relation 1 (sensitivity profile) requires that the response of the six-step forecast to a one-standard-deviation shift of the input window stays inside a band learned offline on the healthy model; the first relation that was tried, equivariance under translation, was falsified by the data because the trained LSTM mean-reverts, and it was replaced by the learned band. Relation 2 (recursive self-consistency) requires that the forecast after the first step equals the forecast made from the window extended by the first forecast value. Tolerances are conformal quantiles (alpha = 0.01, family-wise over three probe windows) calibrated on 840 historical windows, so a model that is already corrupt at start-up can be detected; the probe is read-only and disabled by default.

Fault	Detected (30 probes)	Median score (threshold = 1)
corrupt (NaN)	100.0 %	very large
unit change	100.0 %	2100.6

Fault	Detected (30 probes)	Median score (threshold = 1)
flat-line	100.0 %	362.07
stale output	100.0 %	533.35
bit flip (8 flips)	93.3 %	1.09 (marginal)
bit flip (16 flips)	83.3 %	1.13 (marginal)
scaler	16.7 %	0.81 (missed)

Table 6.9 Metamorphic probe: detection per fault (no golden output, no stored reference)

The false-alarm rate depends on how the probe is calibrated. With calibration on the even windows of all years and testing on the odd windows it was 2.0 percent per probe; with calibration on the years up to 2018 and testing after 2018 it rose to 66.0 percent, because the response band does not survive the post-2018 drift and would need periodic re-calibration. In a cluster test with the golden check switched off, the probe scored healthy windows at the threshold (0.996 to 1.0) and missed the injected eight-flip bit fault (0.996). The honest conclusion is that the probe reliably detects output-level and gross faults (which the envelope, invariants and canary already detect) but not subtle weight corruption, for which the golden-output check remains necessary. It is therefore reported as an experimental and partly negative result, not as a claimed novelty.

6.7 Baselines and Cost-Coverage of the Guard Layers

To check that the guard is better than simple alternatives and to quantify what each layer costs, six detectors were compared on the same injected faults using the real service code: a range check (history minimum and maximum), a 3-sigma z-score against the calendar-month mean, a golden test-vector check in the style of LM-Fix, the conformal envelope alone, envelope with invariants, and the full guard (envelope, invariants and golden). Detection was measured on 40 random history windows and false alarms on 300 healthy windows.

Fault	range	z-score 3 sd	golden	envelope	env + inv	full
bitflip	0.0 %	67.5 %	100.0 %	2.5 %	7.5 %	100.0 %
bitflip_severe	90.0 %	100.0 %	100.0 %	100.0 %	100.0 %	100.0 %
scaler	100.0 %	100.0 %	100.0 %	100.0 %	100.0 %	100.0 %
corrupt	100.0 %	100.0 %	100.0 %	100.0 %	100.0 %	100.0 %
unitchange	100.0 %	100.0 %	100.0 %	100.0 %	100.0 %	100.0 %
flatline	0.0 %	100.0 %	100.0 %	37.5 %	100.0 %	100.0 %
stale	0.0 %	100.0 %	100.0 %	72.5 %	97.5 %	100.0 %
false alarms (healthy)	0.0 %	12.3 %	0.0 %	0.0 %	1.0 %	1.0 %

Table 6.10 Detection of injected faults by simple baselines and by the layers of the guard (percent of windows)

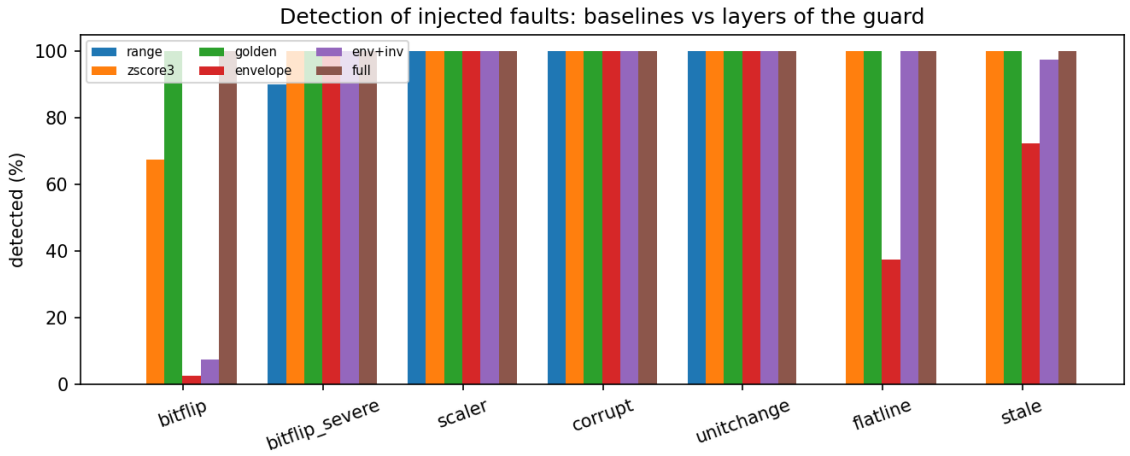


Figure 6.17 Detection of injected faults: baselines versus guard layers

Three honest conclusions follow. First, simple detectors are inadequate: the range check never sees the subtle bit flip, the flat-line or the stale output, and the z-score check raises false alarms on 12.3 percent of healthy windows. Second, the golden test-vector check on its own detected every injected fault with no false alarm, so on a deterministic model it subsumes the other layers for detection; the statement that the layers are complementary therefore holds for per-request checking only. Third, the cost differs by orders of magnitude: a golden check needs a 24-step forecast (182.241 ms) whereas the envelope with invariants needs 0.081 ms. The golden check can therefore be run only periodically (every five seconds it uses about 3.6 percent of one core), while the cheap checks validate every served answer. The resulting design - cheap per-request checks that bound the exposure between golden checks, plus a periodic golden check that bounds the detection latency - is the cost-coverage contribution of the study.

6.8 Healthy-but-Wrong Time and Mean Time to Correctness

Two metrics are defined to describe what availability and restart-time metrics hide. Healthy-but-wrong (HBW) time is the number of seconds during a fault window in which the platform reports the pod Ready while wrong answers are served. Mean time to correctness (MTTC) is the time from injection to the first answer that is not wrong. In the SilentBench data the guard-off configuration (C4) has an HBW time of 30 s (the whole fault window) and an undefined MTTC for every silent fault, which means that the platform never leaves the healthy-but-wrong state; the full guard (G4, H4) has an HBW time of 0 s for every fault; the envelope and invariants (E4) have 0 s except for the subtle bit flip (30 s). Full quality (MTTQ) returns with the operator in 23.8 s (rules) and 21.5 s (bandit) and never without it. The numbers are in results/hbw_mttc.md.

6.9 Compound Faults

Real incidents rarely come alone, so six combinations of a weight or scaler fault with an output-level or delay fault were injected into the real service code (25 requests per scenario; a response is wrong if it deviates from the healthy forecast of the tier it names). Output-level compound faults (flat-line, stale, unit change combined with a bit flip or scaler fault) and corrupt plus delay were handled by all three configurations by serving climatology. The interesting case is a subtle bit flip combined with a slow-model fault: E4 served 100 percent wrong answers from the LSTM with a p95 latency of 2465 ms; G4 served no wrong answer (it fell back to SARIMA) but with a p95 latency of 1615 ms; H4 served no wrong answer with a p95 latency of 768 ms. A subtle silent fault and a slow fault together need all three mechanisms - the golden check, the tier chain and the deadline - and neither of the two faults alone shows this interaction.

Scenario	E4 wrong %	G4 wrong %	H4 wrong %	E4 p95 (ms)	G4 p95 (ms)	H4 p95 (ms)
bitflip+flatline	0.0	0.0	0.0	6	5	3
bitflip+stale	0.0	0.0	0.0	6	3	3
scaler+unitchange	0.0	0.0	0.0	5	3	2
bitflip+delay	100.0	0.0	0.0	2465	1615	768
corrupt+delay	0.0	0.0	0.0	3	7	9
scaler+flatline	0.0	0.0	0.0	6	5	4

Table 6.11 Compound faults: wrong answers and p95 latency per configuration

6.10 Overall Analysis

- The central claim is supported by measurement: default Kubernetes and Compose cannot see silent failures, while a guard that tests the answer converts all injected silent faults into valid answers without false alarms.

- Among per-request checks the layers are complementary; the golden check was added because the others left a measured hole, and on its own it detects every injected fault but costs about 182 ms per check (Section 6.7).

- Automatic restoration needs an operator or a semantic liveness probe; semantic readiness alone only stops traffic.

- The learned recovery policy is not better than simple rules in this action space; its value is that it is derived from data and would choose differently where restarting is expensive (for example crash faults, where the table prefers restart-only at 48 s against 75 s).

- Probe tuning is secondary to redundancy: replicas dominate recovery time.

- Prediction gives useful lead for gradual faults at the cost of false alarms; the shadow

gate separates good and bad candidates cleanly on the tested candidates.

- Several attractive extras gave negative or inconclusive results; recording them prevents over-claiming.

6.11 Summary

The results show a large and statistically significant effect of the proposed mechanisms on the safety of the served answers (0 percent wrong versus 100 percent), on the restoration of full quality (20 of 20 within about 24 seconds versus never) and, with the deadline mechanism, on availability under slow faults (0 percent failed versus 96.8 percent). They also show the limits of the approach, which Chapter 7 summarises.

CHAPTER 7: CONCLUSION

7.1 Conclusion

This project started from a simple but under-studied observation: a machine-learning service on Kubernetes can be perfectly healthy by every standard probe and still serve wrong answers. The work re-engineered the Bangalore Climate Analytics application (SARIMA and a real trained LSTM, unchanged) into a containerised, orchestrated and observable system, built a benchmark of silent faults, and designed and measured a layered set of mechanisms that make the platform aware of the correctness of the answers.

The measurements support five conclusions. First, default Docker Compose and Kubernetes never detect or recover from silent faults: all seven silent faults produced 100 percent wrong answers with the guard off, and nothing was detected in 10 of 10 trials per fault. Second, a window-level conformal envelope with temporal invariants, a cross-model canary and a golden-output integrity check turned every injected silent fault into a valid, labelled answer (0 percent wrong in all fault types, including subtle bit flips that evade the other layers) with zero false alarms in healthy control, in a ten-minute soak and on hold-out windows. Third, automatic restoration of full quality needs a correctness-aware recovery loop: with the operator the service returned to full quality in 20 of 20 trials in about 22 to 24 seconds, whereas it never returned without it, and a cost model that charges for time below full quality was necessary for the learned policy to behave sensibly. Fourth, a deadline- and load-aware degradation mechanism removed the failures caused by slow model calls (96.8 percent to 0 percent failed requests) and reduced the p95 latency under burst load by about four times. Fifth, the extras show that precursor-based prediction gives a useful warning time for gradual faults (69.6 s for memory leaks), that a shadow gate separates good and bad candidate models cleanly, and that the same criterion can drive a standard delivery tool (Argo Rollouts).

The project also produced negative and inconclusive results that are as important as the positive ones: probe tuning is secondary to redundancy, a learned bandit offers no advantage over simple rules in the available action space, log-based detection has no signal for silent faults, the input guard and the LSTM autoencoder generate too many false alarms, and conclusions about autoscaling on a single node could not be drawn.

Recording these results, and withdrawing a claim that a later experiment contradicted, was part of the method.

7.2 Limitations

- Scale: 5 to 10 trials per experimental cell, one model family, one data set, one laptop with a one-node Minikube cluster; confidence intervals are therefore wide for some cells and multi-node effects are not studied.

- The golden-output check detects change from a trusted start-up state; it cannot detect a model that was already wrong when it started, and legitimate retraining requires refreshing the golden outputs after the shadow gate.

- Faults are injected synthetically (weights, scaler, output, delay); real-world incidents may differ. The injected faults were designed by the authors, which can favour the proposed detectors.

- The cross-model canary and the golden check assume deterministic models; stochastic models need statistical variants.

- Most delayed requests end on the climatology tier because all backends share one lock; per-tier locks were not implemented.

- The learned recovery policy has only restart and tier-switch actions, so it cannot outperform the rules policy; its benefit would appear with a richer action set.

- Scaling results (HPA, pre-scaling, KEDA) were obtained on a single node with an earlier thread configuration and are withdrawn as evidence; the RL autoscaler is a simulation only.

- Security: placeholder secrets and unauthenticated fault and shadow endpoints are acceptable only in the lab (see the security review); the GitHub Actions workflow, Trivy scan and keyless signing were written but could not be run on the laptop.

- The metamorphic probe is experimental: marginal against subtle bit flips and sensitive to drift when calibrated on earlier years.

- Novelty claims are limited to the 23 papers reviewed and a limited web search; a structured search of IEEE Xplore and Google Scholar is still required.

7.3 Future Scope

- Improve the metamorphic readiness probe (Section 6.6): periodic re-calibration against drift, a larger and more diverse set of probe windows, and relations with more power

against subtle weight corruption.

- New metrics: healthy-but-wrong time and Mean Time To Correctness (MTTC) as complements to availability and MTTR.
- Per-tier locks and finer admission control for deadline-aware degradation; automatic tuning of the deadline from the latency SLO.
- A multi-node cluster study of autoscaling with warm-up awareness (HPA, KEDA and a learned policy) using the 16-thread image.
- Generalisation to other models and data sets (a second forecasting workload, classification services) and to stochastic models.
- A richer action set for the operator (scale up, roll back to the previous model, switch tier permanently) so that the learned policy can show a benefit.
- Running the CI pipeline with image signing, Trivy scanning and Kyverno verification on GitHub, adding authentication to the control endpoints, and network policies.
- Comparison with a literature baseline for output validation (simple range or threshold detector) and larger numbers of trials for the headline claims.
- A structured literature search and a conference or journal paper based on Appendix B.

7.4 Summary

The study demonstrates, with measurements and with an honest account of its limits, that the self-healing promise of container orchestration can be extended from process health to answer correctness at modest cost, and it provides the code, data and documents needed to reproduce and extend the work.

REFERENCES (IEEE FORMAT)

[1] S. Nigam and S. Patel, "AI-augmented chaos testing: Toward intelligent resilience engineering in cloud-native systems," 2025.

[2] J. Zairi and A. Kenzi, "A comparative study of automatic migration from monolith application to microservices," 2025.

[3] L. Li, "A predictive elastic scaling method for Kubernetes based on load forecasting," 2025.

[4] Z. Sun et al., "A stacking ensemble ML-based failure prediction model for optical networks with imbalanced data," 2024.

[5] X. Wu et al., "Architecture-based failure prediction via LSTM and Bayesian network in service-oriented systems," 2022.

[6] "Autoencoder-based anomaly detection in microservices using distributed tracing," 2024.

[7] M. Zaccarini, F. Poltronieri, D. Borsatti, W. Cerroni, L. Foschini, G. Grabarnik et al., "Chaos engineering based Kubernetes pod rescheduling through deep sets and reinforcement learning," 2025.

[8] D. Singh, "Deployment of seasonal autoregressive integrated moving average (SARIMA) models for network reliability prediction," 2024.

[9] G. Siwach and A. Haridas, "Evaluating operational readiness using chaos engineering simulations on Kubernetes architecture in Big Data," 2022.

[10] V. Berry et al., "Is it worth migrating a monolith to microservices? An experience report on performance, availability and energy usage," 2024.

[11] F. Xue et al., "Kube-CC: Building an experiment education platform for Big Data courses based on Kubernetes," 2023.

[12] L. Yang et al., "MicroMILTS: Fault location for microservices based on mutual information and LSTM autoencoder," 2022.

[13] "Multi-level ensemble anomaly detection for microservices combining statistical analysis, Isolation Forest and adaptive thresholding," 2025.

[14] "Recall-prioritized anomaly detection in cloud microservices using unsupervised transformer autoencoders," 2025.

[15] Z. Wei and K. Bai, "Research on constraint-aware fault recovery diagnosis decision method for offline edge Kubernetes scenarios," 2025.

[16] L.-M. Tufeanu et al., "Residual artifact governance with ML-enabled garbage collection prediction in Kubernetes-based 5G/6G service-based architecture," IEEE Trans. Netw. Service Manag., vol. 23, 2026.

[17] S. Oviya et al., "Robust hybrid autoencoder-LSTM framework for real-time anomaly detection in multi-cloud environments," 2025.

[18] V. P. K. Gummadi and L. S. Chilamkurti, "Service level objective (SLO) observability with Splunk and Dynatrace in microservices," 2025.

[19] C. Shao, X. Zhou et al., "The multi-modal anomaly detection method for microservices based on VGAE-Transformer," 2025.

[20] J. Li, S. Ying, T. Li et al., "TraceDAE: Trace-based anomaly detection in microservice systems via dual autoencoder," IEEE Trans. Netw. Service Manag., vol. 22, no. 5, 2025.

[21] E. Hajihashemi and Y. Shen, "Enhanced multi-model online conformal prediction."

[22] Z. Zhang, C. Wan et al., "SGCAD: Structure-aware Granger causality-based anomaly detection for microservice systems."

[23] LM-Fix: Lightweight bit-flip detection and rapid recovery framework for language models, arXiv:2511.02866.

[24] WATCH: Adaptive monitoring for AI deployments via weighted-conformal martingales, arXiv:2505.04608.

[25] Cloud-native evaluation-as-a-service: A microservices architecture for scalable AI monitoring with conformal guarantees, arXiv:2607.21623.

[26] Trusted floors under untrusted learners: A runtime assured-SLO guard for ML serving, arXiv:2607.09992.

[27] Self-healing machine learning: A framework for autonomous adaptation in real-world environments, arXiv:2411.00186.

[28] Enhancing reliability in AI inference services: An empirical study on real production incidents, arXiv:2511.07424.

[29] RADAR: Run-time adversarial weight attack detection and accuracy recovery, in Proc. DATE, 2021.

[30] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735-1780, 1997.

[31] G. E. P. Box, G. M. Jenkins, G. C. Reinsel and G. M. Ljung, Time Series Analysis: Forecasting and Control. Wiley.

[32] V. Vovk, A. Gammerman and G. Shafer, Algorithmic Learning in a Random World. Springer, 2005.

[33] A. N. Angelopoulos and S. Bates, "A gentle introduction to conformal prediction and distribution-free uncertainty quantification," arXiv:2107.07511, 2021.

[34] I. Gibbs and E. Candes, "Adaptive conformal inference under distribution shift," in Advances in Neural Information Processing Systems (NeurIPS), 2021.

[35] F. T. Liu, K. M. Ting and Z.-H. Zhou, "Isolation forest," in Proc. IEEE Int. Conf. Data Mining (ICDM), 2008.

[36] P. Auer, N. Cesa-Bianchi and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," Machine Learning, vol. 47, pp. 235-256, 2002.

[37] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," J. Amer. Statist. Assoc., vol. 22, no. 158, pp. 209-212, 1927.

[38] B. Burns, B. Grant, D. Oppenheimer, E. Brewer and J. Wilkes, "Borg, Omega, and Kubernetes," ACM Queue, vol. 14, no. 1, 2016.

[39] B. Beyer, C. Jones, J. Petoff and N. R. Murphy (eds.), Site Reliability Engineering: How Google Runs Production Systems. O'Reilly Media, 2016.

[40] The Kubernetes Authors, "Kubernetes documentation: Configure liveness, readiness and startup probes." [Online]. Available: https://kubernetes.io/docs/

[41] Prometheus Authors, "Prometheus documentation." [Online]. Available: https://prometheus.io/docs/

[42] OpenTelemetry Authors, "OpenTelemetry documentation." [Online]. Available: https://opentelemetry.io/docs/

Note: the first twenty-three entries are the papers supplied as the reading list; authors, years and venues are those visible on the first page or in the title and must be completed from IEEE Xplore before submission to a conference or journal.

APPENDIX A DATASET SAMPLE

First rows of the daily data file (data.csv) and a sample of rows from the year 2018, as read by the application.

Date	Rain	Temp Max	Temp Min	Humidity
1951-01-01	0	26.96999931	15.64999962	77.36
1951-01-02	0	26.18000031	14.68999958	77.02
1951-01-03	0	26.30999947	14.94999981	77.28
1951-01-04	0	26.72999954	14.84000015	76.22
1951-01-05	0	26.93000031	14.52999973	75.2
1951-01-06	0	26.25	13.48999977	74.48
1951-01-07	0	26.45000076	12.69999981	72.5
1951-01-08	0	25.69000053	15.81000042	80.24
1951-01-09	0	25.88999939	15.27999973	78.78
1951-01-10	0	26.62000084	15.27000046	77.3
1951-01-11	0	27.04999924	15.14999962	76.2
1951-01-12	0	27.77000046	14.77000046	74.0
1951-01-13	0	28.21999931	15.93999958	75.44
1951-01-14	0	27.87000084	16.62000084	77.5
1951-01-15	0	27.90999985	16.47999954	77.14

Table A.1 First 15 rows of data.csv

Date	Rain	Temp Max	Temp Min	Humidity
2018-07-01	0	30.73396873	20.98983192	80.51
2018-07-02	0	31.81499672	20.99389076	78.36
2018-07-03	1.42614603	29.90728951	20.82320786	81.83
2018-07-04	0	30.07770729	20.89439964	81.63
2018-07-05	0	30.64569473	20.9339695	80.58
2018-07-06	0	29.32722092	20.76373863	82.87
2018-07-07	2.130088329	28.06831741	21.24263954	86.35
2018-07-08	1.429886937	27.53249168	20.8086853	86.55
2018-07-09	0	28.70767212	20.79549217	84.18
2018-07-10	1.683328152	27.98830795	20.7418766	85.51

Table A.2 Ten rows from July 2018

The file contains 26,835 daily rows from 1951-01-01 to 2024-06-20.

APPENDIX B RESEARCH PAPER (IEEE FORMAT)

The following is a condensed draft of the research paper that summarises this project. It is written in a single column here; the IEEE two-column template should be applied when it is submitted.

Correctness-Aware Self-Healing for ML Forecasting Services on Kubernetes: Detecting and Recovering from Silent Failures

Abstract - Container orchestrators restart services whose process fails, but a machine-learning service can fail silently: it keeps answering HTTP 200 with wrong values, so default liveness and readiness probes never react. We study this blind spot with a Kubernetes-hosted climate forecasting service (SARIMA and LSTM) and a silent-fault benchmark of eight injected faults. With default probes every silent fault produced 100 percent wrong answers and none was detected (10 trials per cell). We propose a layered correctness guard - a window-level split-conformal envelope, temporal invariants, a cross-model canary and a golden-output integrity check - with tiered graceful degradation, a correctness SLO, a Kubernetes operator that restores full quality, a precursor-based failure predictor, a shadow-traffic gate and deadline- and load-aware degradation. The guard served 0 percent wrong answers for all faults, including subtle bit flips that evade the other layers, with no false alarms in healthy control; the operator restored full quality in 20 of 20 trials in about 22-24 s against never without it; and deadline-aware degradation reduced failed requests under a delay fault from 96.8 percent to 0 percent. We also report negative results (probe tuning, learned policy, log-based detection, scaling) and the limits of the study.

Index Terms - silent failures, Kubernetes, self-healing, conformal prediction, MLOps, graceful degradation, SLO.

I. Introduction

Orchestration platforms equate health with process liveness. For ML services this is insufficient: damaged weights, a wrong scaler, a stale cache or a unit change leaves the process and the HTTP interface intact. The consequence is a state we call healthy-but-wrong. Prior work on Kubernetes resilience injects infrastructure faults and detects anomalies in metrics, logs and traces; we found no study in a set of 23 recent IEEE papers that defines health on the model output. This paper contributes (1) a measured characterisation of the blind spot (SilentBench), (2) a layered correctness guard with controlled false-alarm rate, (3) correctness-aware degradation, SLO and recovery, (4) prediction and delivery safeguards, and (5) a deadline- and load-aware degradation mechanism.

II. Related Work

Chaos-engineering studies on Kubernetes measure recovery from pod or node faults and, more recently, use reinforcement learning and digital twins to orchestrate experiments or reschedule pods. Anomaly detectors based on autoencoders, graph models and transformers operate on traces, logs and metrics. Failure-prediction work forecasts reliability or time to collapse from infrastructure data. SLO practice defines objectives on latency and availability. Conformal

prediction gives distribution-free guarantees and has been studied online and with multiple models, but not as a health check for deployed services. None of these evaluates correctness-aware health checking of ML outputs.

III. System and Method

A. System. A Flask frontend, a Flask/gunicorn inference API (SARIMA, LSTM, climatology, cache tiers), PostgreSQL, Prometheus, Grafana, Jaeger and a kopf operator run on a one-node Minikube cluster. Data: 26,835 daily Bengaluru weather records (1951-2024) aggregated to monthly series.

B. Guard. For each window length the family-wise score of historical windows in the calibration period (up to 2018) is computed and its (1 - alpha) empirical quantile (alpha = 0.01) normalises the score of a served forecast; a forecast is valid if the score is at most 1. Temporal invariants (non-constant mean temperature) are added. A background canary runs all models every 5 s, aligns them by calendar month and marks a disagreeing model suspect. A golden-output check compares each forecast with the forecast stored at start-up (tolerance 0.002 history standard deviations). The first tier that passes every check answers (LSTM, SARIMA, climatology, cache).

C. Recovery and delivery. The operator evaluates /health/semantic every 5 s and restarts the deployment after three consecutive invalid checks; a digital-twin cost table with a quality term drives a UCB bandit policy. A precursor predictor (Isolation Forest, Mahalanobis) warns of leaks and drift. A shadow gate promotes a candidate model only if its reject ratio is at most 5 percent over at least 20 mirrored requests.

D. Deadline- and load-aware degradation. Each request has an 800 ms budget; each tier runs in a worker pool under a bounded semaphore (2 LSTM, 3 SARIMA); a tier that cannot answer in time is shed and the next tier answers.

IV. Experiments and Results

SilentBench injects eight faults (corrupt, bit flip, severe bit flip, scaler, stale, unit change, flat-line, delay) into configurations C4 (guard off), D4 (envelope), E4 (envelope and invariants), G4 (plus golden check) and H4 (plus deadline) at 5 requests per second for 30 s, with 5-10 trials per cell. With the guard off, every silent fault gave 100 percent wrong answers and 0 detections. D4 and E4 converted all faults except the subtle bit flip into valid answers; the flat-line fault required the invariants. G4 caught the subtle bit flip in 10 of 10 trials (Fisher exact p = 1.1e-5 versus C4) and served 0 percent wrong answers for every fault, with 100 percent correct answers in healthy control. Offline, the false-alarm rate on hold-out windows was 0.0 percent at alpha = 0.01 (20-36.7 percent at alpha = 0.05).

Operator: without an operator full quality never returned (0/20); with the rules policy it returned in 23.6 s (95 percent CI 22.3-24.9; 20/20); the first bandit version restored 5/20, the cost-aware version 20/20 in 21.6 s. Predictor: leak lead 69.6 s (6/6) versus 23.9 s for a static memory threshold, drift lead 23.7 s (6/6), 2.45 false alarms per 10 min. Shadow gate: 15/15 bad candidates rejected, 6/6 good promoted. Deadline-aware degradation: under the delay fault failed requests fell from 96.8 to 0 percent and p95 from 2008 to 762 ms; burst p95 fell about fourfold.

V. Discussion and Limitations

The layers are complementary: each closed a class of faults left by the previous layers. The golden check relies on determinism and a trusted start-up state. Probe tuning was secondary to replicas; the learned policy equalled rules in a two-action space; log-based detection had no signal; scaling results on one node were inconclusive and an earlier negative claim was withdrawn after a thread-pool artefact was found. Sample sizes are small and faults are synthetic.

VI. Conclusion

Correctness-aware health checking closes the silent-failure blind spot of container orchestration at modest cost. Future work includes metamorphic probes that need no stored reference, new metrics (healthy-but-wrong time, mean time to correctness), multi-node scaling studies and a structured literature search.